



Digital System Design

Lecture Note: icyice

Illustration: ©SEGA ©Colorful Palette

©Crypton Future Media

目录

1	逻辑代数	1
§1.1	逻辑代数基础	1
§1.2	逻辑化简	3
2	组合逻辑	9
§2.1	基本组合电路单元	9
§2.2	运算器	15
§2.3	组合设计	17
3	时序逻辑	19
§3.1	触发器	19
§3.2	时序电路	23
4	控制逻辑	33
§4.1	存储器	33
§4.2	控制器	34
§4.3	微程序控制	41
5	门电路	43
§5.1	TTL	43
§5.2	CMOS	50
6	脉冲电路	55
§6.1	触发器电路	55
§6.2	多谐振荡器	57
§6.3	555 定时器	60
7	测试电路	63
8	微处理器	66
§8.1	控制单元与数据通路	66
§8.2	机器语言与汇编语言	67
9	Verilog 语言	69
§9.1	Verilog	69
§9.2	逻辑综合	73

1 逻辑代数

§1.1 逻辑代数基础

定义 1.1 布尔代数

用简单 (0/1) 代数描述复杂性事物。



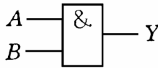
布尔代数具有三种基本逻辑。

定义 1.2 与 (AND)

条件同时具备，结果发生。



$$Y = A \wedge B = A \& B = AB$$



与

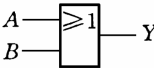


定义 1.3 或 (OR)

条件之一具备，结果发生。



$$Y = A \vee B = A + B$$



或

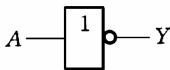


定义 1.4 非 (NOT)

条件不具备，结果发生。



$$Y = \neg A = A'$$



非





注释:

关于基本逻辑，可以参考离散数学课程。

定义 1.5 与非 (NAND)

$$Y = (A \cdot B)'$$



定义 1.6 或非 (NOR)

$$Y = (A + B)'$$



定义 1.7 与或非 (AND - NOR)

$$Y = (A \cdot B + C \cdot D)'$$



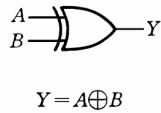
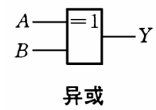
除了上述三种复合运算以外，还有以下两种运算。

定义 1.8 异或 (XOR)

条件不相同，结果发生。



$$Y = A \oplus B$$



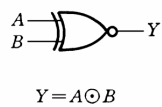
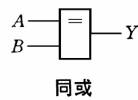
A	B	Y
0	0	0
0	1	1
1	0	1
1	1	0

定义 1.9 同或 (XAND)

条件相同，结果发生。



$$Y = A \odot B$$



A	B	Y
0	0	1
0	1	0
1	0	0
1	1	1

对于逻辑运算，有以下几个定理。

定理 1.1

逻辑代数满足交换律、结合律和分配律。



**注意：**

加法对乘法的分配： $A + BC = (A + B)(A + C)$ 。

定理 1.2 反演律

$$(AB)' = A' + B'$$

$$(A + B)' = A'B'$$

**注释：**

也叫德摩根（De Morgan）定律。

定理 1.3 吸收律

$$A + AB = A$$

$$A + A'B = A + B$$

**§1.2 逻辑化简****定理 1.4 代入定理**

在任何一个包含 A 的逻辑等式中，若以另外一个逻辑式代入式中 A 的位置，则等式依然成立。

**例题 1.1**

初始逻辑式：

$$A + BC = (A + B)(A + C)$$

进行代入替换：

$$\begin{aligned} A + B(CD) &= (A + B)(A + CD) \\ &= (A + B)(A + C)(A + D) \end{aligned}$$

定理 1.5 反演定理

对于任意逻辑式 Y，其否定形式 Y'：

$\cdot \Rightarrow +, + \Rightarrow \cdot, 0 \Rightarrow 1, 1 \Rightarrow 0,$

原变量 $\Rightarrow$ 反变量

反变量 $\Rightarrow$ 原变量



**注意:**

变换顺序先括号，然后乘，最后加；不属于单个变量的上的反号保留不变。

**例题 1.2**

原逻辑表达式:

$$Y = A(B + C) + CD$$

其反演形式:

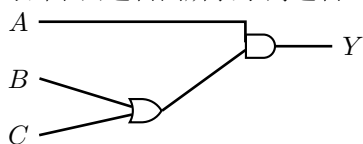
$$\begin{aligned} Y' &= (A' + B'C')(C' + D') \\ &= A'C' + B'C' + A'D' + B'C'D' \\ &= A'C' + B'C' + A'D' \end{aligned}$$

定义 1.10 逻辑函数

若以逻辑变量为输入，运算结果为输出，则输入变量值确定以后，输出的取值也随之而定。输入/输出之间是一种函数关系，称为逻辑函数。

**例题 1.3**

写出下面逻辑图所表示的逻辑函数。



解: $Y = A \cdot (B + C)$

定义 1.11 最小项

最小项 m :

- m 是乘积项
- 包含 n 个因子
- n 个变量均以原变量和反变量的形式在 m 中出现一次
- 编号



两个相邻的最小项（只有一个变量不同的两个最小项）之和可以合并，消去一对因子，只留下公共因子。

**注释:**

最小项的编号规则如下：

最小项	取值	对应	编号
	ABC	十进制数	
$A'B'C'$	0 0 0	0	m_0
$A'B'C$	0 0 1	1	m_1
$A'BC'$	0 1 0	2	m_2
$A'BC$	0 1 1	3	m_3
$AB'C'$	1 0 0	4	m_4
$AB'C$	1 0 1	5	m_5
ABC'	1 1 0	6	m_6
ABC	1 1 1	7	m_7

定理 1.6 最小项的性质

- 在输入变量任一取值下，有且仅有一个最小项的值为 1
- 全体最小项之和为 1
- 任何两个最小项之积为 0
- 两个相邻的最小项之和可以合并，消去一对因子，只留下公共因子



对于任何逻辑变量， $A + A' = 1$ ， $AA' = 0$ 。



例题 1.4

化简逻辑表达式：

$$\begin{aligned} Y(A, B, C) &= ABC' + BC \\ &= ABC' + BC(A + A') \\ &= ABC' + ABC + A'BC \\ &= \sum m(3, 6, 7) \end{aligned}$$

定义 1.12 最大项

最大项 M ：

- M 是相加项
- 包含 n 个因子
- n 个变量均以原变量和反变量的形式在 M 中出现一次



注释：

最大项的编号规则如下：

最大项	取值	对应	编号
	ABC	十进制数	
$A' + B' + C'$	1 1 1	7	M_7
$A' + B' + C$	1 1 0	6	M_6
$A' + B + C'$	1 0 1	5	M_5
$A' + B + C$	1 0 0	4	M_4
$A + B' + C'$	0 1 1	3	M_3
$A + B' + C$	0 1 0	2	M_2
$A + B + C'$	0 0 1	1	M_1
$A + B + C$	0 0 0	0	M_0

定理 1.7

若 $Y = \sum m_i$, 则

$$Y = \prod_{i \neq k} m'_k = \prod_{i \neq k} M_k$$



例题 1.5

化简逻辑表达式:

$$\begin{aligned}
 Y &= AC + B'C + BD' + CD' + A(B + C') + A'BCD' + AB'DE \\
 &= AC + B'C + BD' + CD' + A(\overline{B + C'}) + AB'DE \\
 &= AC + B'C + BD' + CD' + \overline{AB'C} + AB'DE \\
 &= AC + B'C + BD' + CD' + A + AB'DE \\
 &= A + B'C + BD' + CD' \\
 &= A + B'C + BD'
 \end{aligned}$$

公式化简的结果通常不是唯一的。

定义 1.13 卡诺图

将逻辑函数的最小项之和以图形的方式表示出来的图叫做卡诺图。



注释:

四变量的卡诺图:

		CD			
AB		00	01	11	10
	00	m_0	m_1	m_3	m_2
	01	m_4	m_5	m_7	m_6
	11	m_{12}	m_{13}	m_{15}	m_{14}
	10	m_8	m_9	m_{11}	m_{10}

在卡诺图中，最小项的相邻性可以从图形中直观地反映出来，从而可以进行合并消去。

定理 1.8 卡诺图消元定理

- 两个相邻最小项可合并为一项，消去一对因子
- 四个排成矩形的相邻最小项可合并为一项，消去两对因子
- 八个相邻最小项可合并为一项，消去三对因子



注意：

最左边和最右边在逻辑上也是相邻项，可以相消合并，并且卡诺图的最小项可以多次利用，但要求矩形的数目最少、大小最大。



例题 1.6

化简逻辑表达式：

$$Y = ABC + ABD + AC'D + C' \cdot D' + AB'C + A'CD'$$

		CD			
AB		00	01	11	10
	00	1	0	0	1
	01	1	0	0	1
	11	1	1	1	1
	10	1	1	1	1

化简结果：

$$A + D'$$

定义 1.14 无关项

约束项和任意项可以写入函数式，也可不包含在函数式中，统称为无关项

可以充分利用无关项化简逻辑表达式。



例题 1.7

化简逻辑表达式:

$$Y(A, B, C, D) = \sum m(2, 4, 6, 8)$$

约束条件: $m_5 + m_{10} + m_{11} + m_{12} + m_{13} + m_{14} + m_{15} = 0$

		CD			
		00	01	11	10
AB	00	0	0	0	1
	01	1	x	0	1
	11	x	x	x	x
	10	1	0	x	x

化简结果:

$$Y = AD' + BD' + CD'$$

定义 1.15 格雷码

在表示一组数字时, 任何两个相邻的代码 (以及最大数和最小数之间) 只有一位二进制数不同。



注释:

4 位格雷码和二进制的对应表如下

Decimal	Binary	Hexadecimal	BCD	GRAY
0	0	0	0000	0000
1	1	1	0001	0001
2	10	2	0010	0011
3	11	3	0011	0010
4	100	4	0100	0110
5	101	5	0101	0111
6	110	6	0110	0101
7	111	7	0111	0100
8	1000	8	1000	1100
9	1001	9	1001	1101
10	1010	A	0001 0000	1111
11	1011	B	0001 0001	1110
12	1100	C	0001 0010	1010
13	1101	D	0001 0011	1011
14	1110	E	0001 0100	1001
15	1111	F	0001 0101	1000

2 组合逻辑

§2.1 基本组合电路单元

组合电路的特点：

- 任意时刻的输出仅取决于该时刻的输入，没有反馈
- 不含存储单元

定义 2.1 编码器

将输入的每个高/低电平信号变成一个对应的二进制代码。

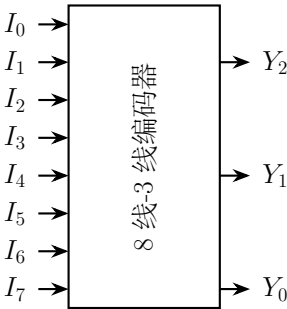


普通编码器：任何时刻只允许输入一个编码信号



例题 2.1

3 位二进制普通编码器



输入输入如下

输 入								输 出		
I_0	I_1	I_2	I_3	I_4	I_5	I_6	I_7	Y_2	Y_1	Y_0
1	0	0	0	0	0	0	0	0	0	0
0	1	0	0	0	0	0	0	0	0	1
0	0	1	0	0	0	0	0	0	1	0
0	0	0	1	0	0	0	0	0	1	1
0	0	0	0	1	0	0	0	1	0	0
0	0	0	0	0	1	0	0	1	0	1
0	0	0	0	0	0	1	0	1	1	0
0	0	0	0	0	0	0	1	1	1	1

优先编码器：允许同时输入两个以上的编码信号，但只对其中优先权最高的一个进行编码

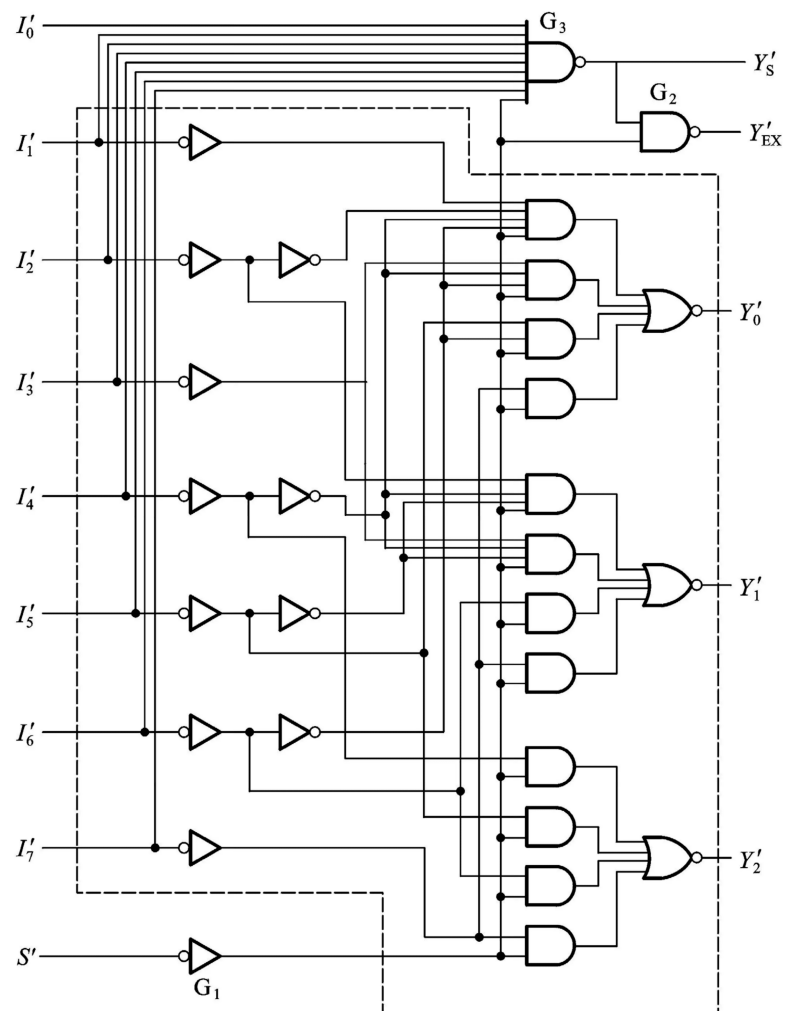


例题 2.2

8 线-3 线优先编码器

输 入								输 出		
I_0	I_1	I_2	I_3	I_4	I_5	I_6	I_7	Y_2	Y_1	Y_0
X	X	X	X	X	X	X	1	1	1	1
X	X	X	X	X	X	1	0	1	1	0
X	X	X	X	X	1	0	0	1	0	1
X	X	X	X	1	0	0	0	1	0	0
X	X	X	1	0	0	0	0	0	1	1
X	X	1	0	0	0	0	0	0	1	0
X	1	0	0	0	0	0	0	0	0	1
1	0	0	0	0	0	0	0	0	0	0

74HC148 优先编码器：



输入输出如下：

输 入									输 出				
S'	I'_0	I'_1	I'_2	I'_3	I'_4	I'_5	I'_6	I'_7	Y'_2	Y'_1	Y'_0	Y'_S	Y'_{EX}
1	X	X	X	X	X	X	X	X	1	1	1	1	1
0	1	1	1	1	1	1	1	1	1	1	1	0	1
0	X	X	X	X	X	X	X	0	0	0	0	1	0
0	X	X	X	X	X	X	0	1	0	0	1	1	0
0	X	X	X	X	X	0	1	1	0	1	0	1	0
0	X	X	X	X	0	1	1	1	0	1	1	1	0
0	X	X	X	0	1	1	1	1	1	0	0	1	0
0	X	X	0	1	1	1	1	1	1	0	1	1	0
0	X	0	1	1	1	1	1	1	1	1	0	1	0
0	0	1	1	1	1	1	1	1	1	1	1	1	0

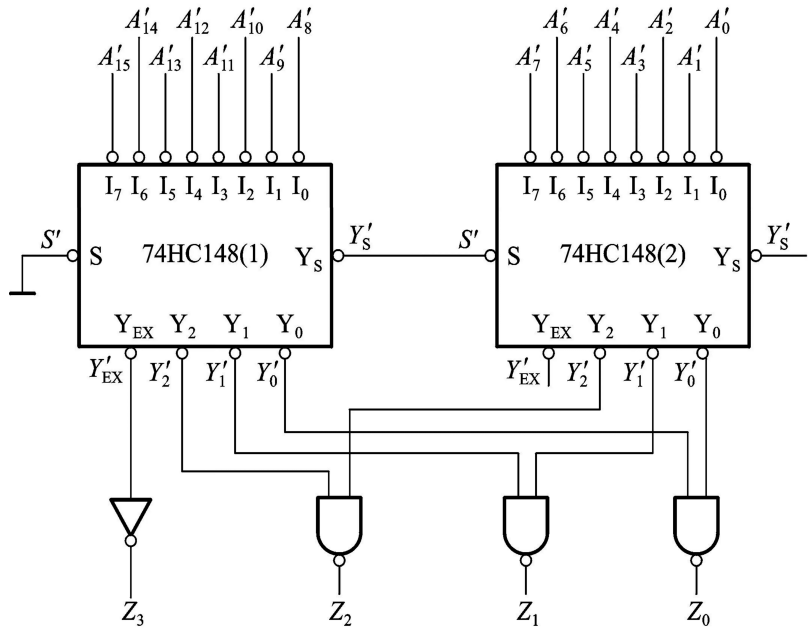


注释:

附加输出信号的状态及含义:

Y'_S	Y'_{EX}	状态
1	1	不工作
0	1	工作, 但无输入
1	0	工作, 且有输入
0	0	不可能出现

74HC148 的级联:





注意：

- 第一片为高优先权
- 只有 (1) 无编码输入时，(2) 才允许工作
- 第 (1) 片 $Y'_{EX} = 0$ 时表示对 $A'_{15} \sim A'_8$ 的编码
- 低 3 位输出应是两片的输出的“或”

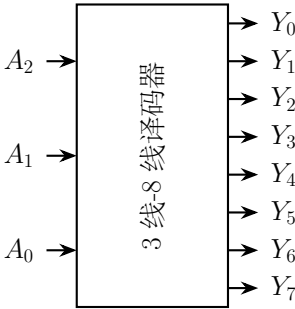
定义 2.2 译码器

将每个输入的二进制代码译成对应的输出高、低电平信号。



例题 2.3

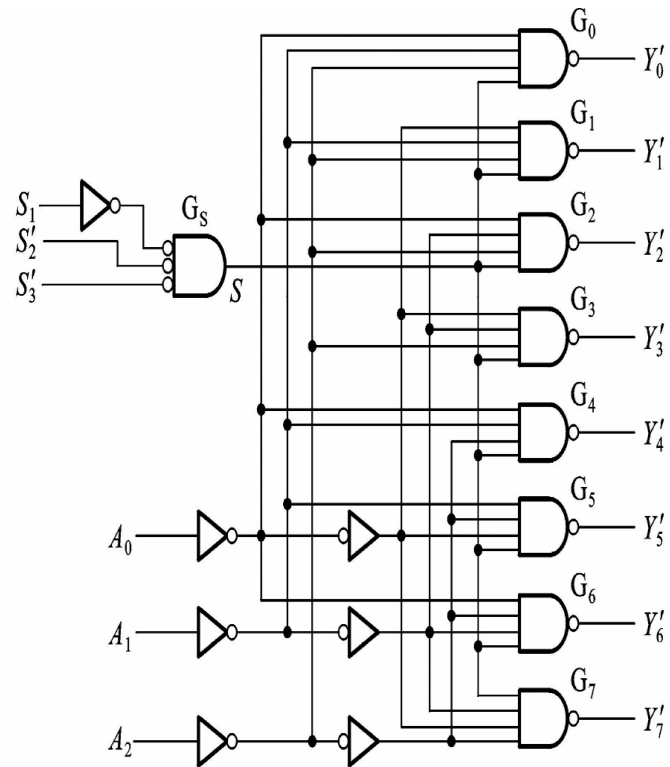
3 线—8 线译码器



输入输出如下：

输 入			输 出							
A_2	A_1	A_0	Y_7	Y_6	Y_5	Y_4	Y_3	Y_2	Y_1	Y_0
0	0	0	0	0	0	0	0	0	0	1
0	0	1	0	0	0	0	0	0	1	0
0	1	0	0	0	0	0	0	1	0	0
0	1	1	0	0	0	0	1	0	0	0
1	0	0	0	0	0	1	0	0	0	0
1	0	1	0	0	1	0	0	0	0	0
1	1	0	0	1	0	0	0	0	0	0
1	1	1	1	0	0	0	0	0	0	0

74HC138 集成译码器：



输入输出如下：

输 入					输 出							
S_1	$S_2' + S_3'$	A_2	A_1	A_0	Y_7'	Y_6'	Y_5'	Y_4'	Y_3'	Y_2'	Y_1'	Y_0'
0	X	X	X	X	1	1	1	1	1	1	1	1
X	1	X	X	X	1	1	1	1	1	1	1	1
1	0	0	0	0	1	1	1	1	1	1	1	0
1	0	0	0	1	1	1	1	1	1	1	0	1
1	0	0	1	0	1	1	1	1	1	0	1	1
1	0	0	1	1	1	1	1	1	0	1	1	1
1	0	1	0	0	1	1	1	0	1	1	1	1
1	0	1	0	1	1	1	0	1	1	1	1	1
1	0	1	1	0	1	0	1	1	1	1	1	1
1	0	1	1	1	0	1	1	1	1	1	1	1

同 74HC148 一样，74HC138 也可以进行级联，这里不过多叙述。

将 n 位二进制译码输出的最小项组合起来，可获得任何形式的输入变量不大于 n 的组合函数

$$Y = \sum_i m_i$$



例题 2.4

利用 74HC138 设计一个多输出的组合逻辑电路，输出逻辑函数式为：

$$Z_1 = AC' + A'BC + AB'C$$

$$Z_2 = BC + A'B'C$$

$$Z_3 = A'B + AB'C$$

$$Z_4 = A'BC' + B'C' + ABC$$

解：

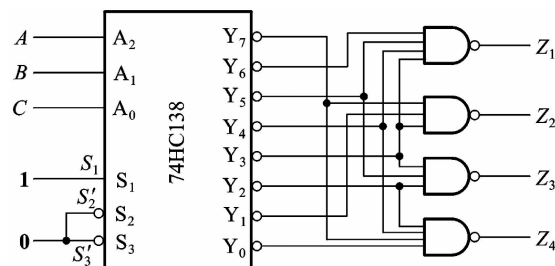
$$Z_1 = \sum m(3, 4, 5, 6) = (m'_3 m'_4 m'_5 m'_6)'$$

$$Z_2 = \sum m(1, 3, 7) = (m'_1 m'_3 m'_7)'$$

$$Z_3 = \sum m(2, 3, 5) = (m'_2 m'_3 m'_5)'$$

$$Z_4 = \sum m(0, 2, 4, 7) = (m'_0 m'_2 m'_4 m'_7)'$$

设计电路如下：



注意：

注意使能端的高低电平，以及输入输出引脚之间的关系；在输入/输出带'表示低电平有效，如 74HC148 的输入为 0 时激活，输出为正常输出的取反（如 7 的输出为 000，0 的输出为 111）。



小结：

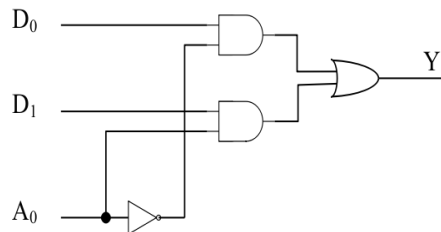
设计译码器的基本原理：

- 一个译码器提供 n 个输入变量的 2^n 个最小项，译码器的输出由每一组输入唯一确定
- 任何布尔函数可以表示成最小项之和
- 任何组合电路由 n 个输入， m 个输出可用 n -to- 2^n 译码器和 m 个或门实现

§2.2 运算器

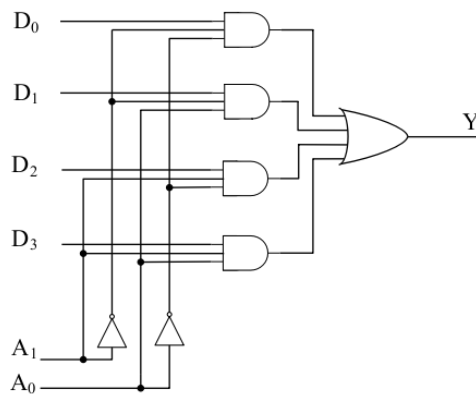
定义 2.3 数据选择器

数据选择器是从多路输入线中选择其中的一路到输出线的一种组合电路。



例题 2.5

四选一选择器 (mux4to1)



真值表如下

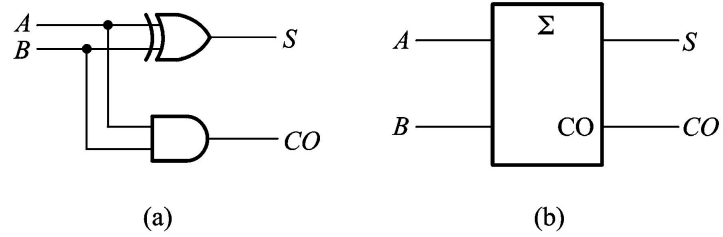
A_1	A_0	Y
0	0	D_0
0	1	D_1
1	0	D_2
1	1	D_3

具有 $n-1$ 位地址输入的数据选择器，可实现 n 个变量布尔函数；数据选择器就是一个带或（OR）门的译码器。

定义 2.4 半加器

不考虑来自低位的进位，将两个 1 位的二进制数相加。

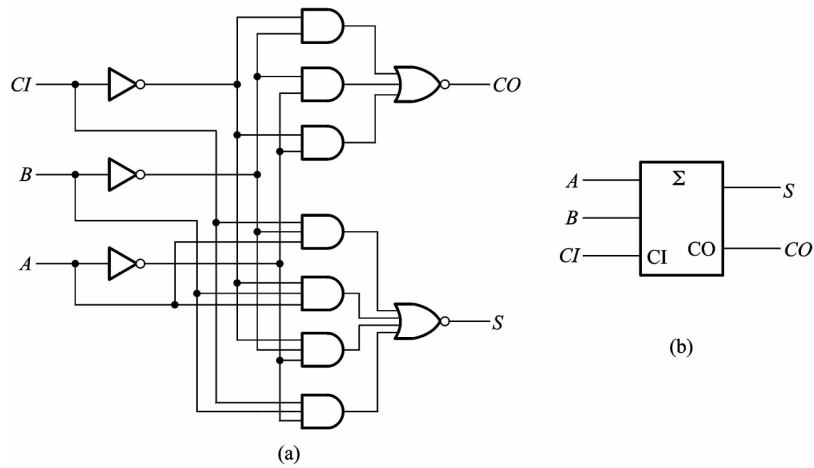




$$S = A \oplus B, CO = AB$$

定义 2.5 全加器

将两个 1 位二进制数 A,B 及来自低位的进位 CI 相加。

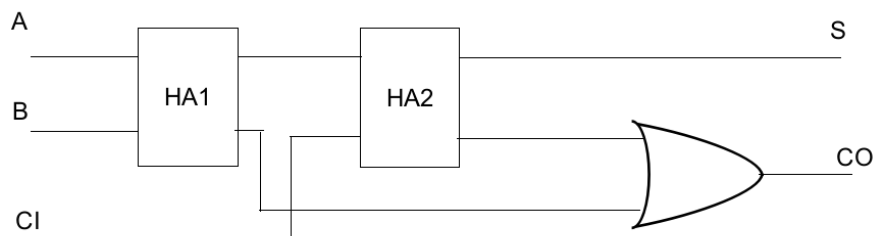


$$S = A \oplus B \oplus CI, CO = (A \cdot B + (CI \cdot (A \oplus B)))$$



注释：

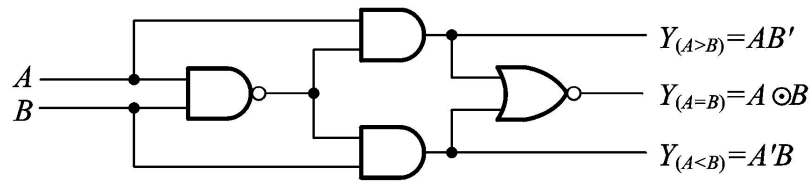
可以用两个半加器实现全加器：



定义 2.6 数值比较器

用来比较两个二进制数的数值大小。





多位数值比较器：从高位比起，只有高位相等，才比较下一位。



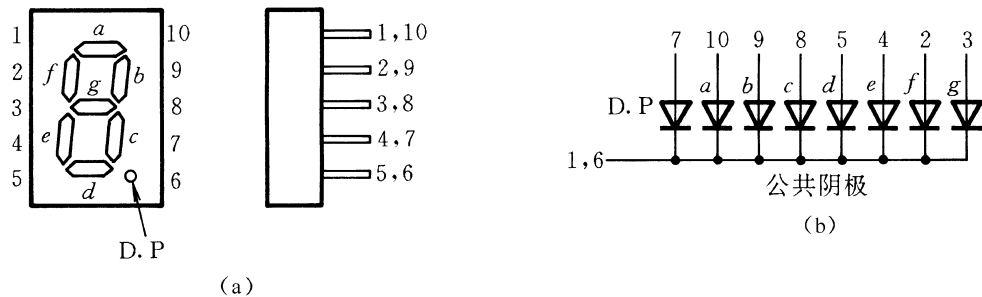
注意：

如果只需要比较三位的话，空余的一位请小心处理，避免不必要的麻烦。

最标准的做法是将最高位置零，避免因数值改变带来的问题。

§2.3 组合设计

显示译码器：



定义 2.7 竞争-冒险

两个输入“同时向相反的逻辑电平变化”，称存在“竞争”。

因“竞争”而可能在输出产生尖峰脉冲的现象，称为“竞争-冒险”。



消除竞争-冒险的方法：

- 接入滤波电容：尖峰脉冲很窄，用很小的电容就可将尖峰削弱到 V_{TH} 以下
- 引入选通脉冲：取选通脉冲作用时间，在电路达到稳定之后，P 的高电平期的输出信号不会出现尖峰
- 修改逻辑设计



例题 2.6

考虑下面的逻辑表达式

$$Y = AB + A'C$$

在 $B = C = 1$ 的条件下， $Y = A + A' \Rightarrow$ 稳态下 $Y = 1$

当 A 改变状态时存在竞争—冒险，为消除竞争—冒险可将逻辑改为

$$Y = AB + A'C + BC$$



例题 2.7

用一个 3 线-8 线译码器和门电路设计逻辑函数：

$$Z_1 = AB + AC$$

$$Z_2 = \overline{AB + BD} \cdot BC + \overline{AB} + \overline{BC}$$

解：

$$Z_1 = ABC + AB\overline{C} + A\overline{B}C = m_4 + m_6 + m_7 = \overline{m_4} \cdot \overline{m_6} \cdot \overline{m_7}$$

$$\begin{aligned} Z_2 &= \overline{AB + BD} \cdot BC + \overline{AB} + \overline{BC} = ABC\overline{D} + \overline{AB} + \overline{BC} \\ &= ABC\overline{D} + \overline{A}BC + \overline{A}\overline{B}C + AB\overline{C} = m_2 + m_3 + m_6 + m_7\overline{D} \\ &= \overline{m_2} \cdot \overline{m_3} \cdot \overline{m_6} \cdot \overline{m_7\overline{D}} = \overline{m_2} \cdot \overline{m_3} \cdot \overline{m_6} \cdot (\overline{m_7} + D) \end{aligned}$$



小结：

本章主要内容：

- 编码器与译码器
- 数据选择器
- 半加器与全加器
- 数据比较器

分析方法：

- 明确题目的输入输出要求
- 根据题目条件列真值表
- 通过卡诺图等方式进行逻辑化简
- 使用基本组合电路模块进行逻辑组合

在考试中，如果没给芯片的具体型号，则一般只用考虑输入输出的逻辑设计，不用在意具体的芯片细节。

3 时序逻辑

§3.1 触发器

定义 3.1 时序电路

带有正反馈的电路。



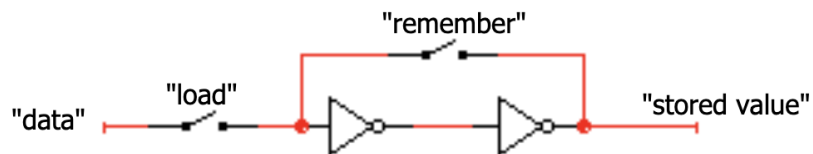
时序电路基本分为两大类：

- 电平敏感锁存器 Latches：使用信号级别而不是信号转换操作的存储单元称为锁存器
- 边沿触发器 Flip-flops：触发器是一种能够存储一位信息的二进制存储器件

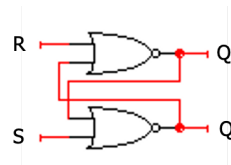
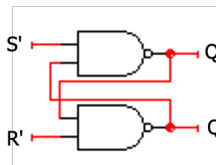


注释：

最简单的反馈电路就是由两个反相器构成的反馈回路：



最简单的触发器是由两个与非门或两个或非门交叉耦合构成的触发器：



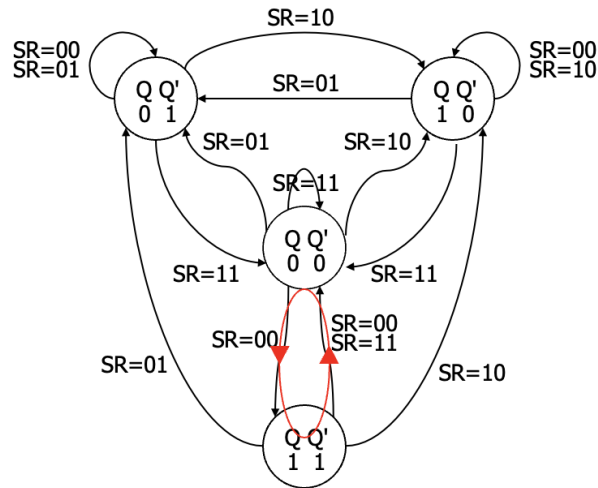
注意：

注意与非门触发器和或非门触发器的逻辑不同：

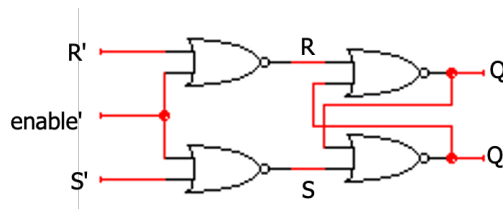
- 与非门中， S' 对应 Q ， S 对应 Q'
- 或非门中， R 对应 Q ， S 对应 Q'

一般约定 S 为置位端， R 为复位端；禁止与非门输入同时为 0，或非门输入同时为 1（同时改变后会引起输出的不确定）。

以 RS 触发器为例，我们可以画出下面的状态图：



为了防止轻微毛刺对输出带来的影响，我们可以采用门控锁存：



定义 3.2 时钟

时钟是有规律的周期信号。

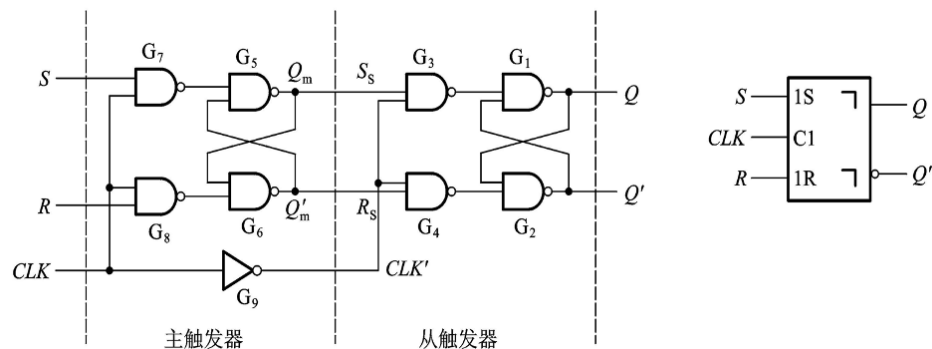
- 周期：高低电平周期转换的时间
- 占空比：高电平部分占周期的比例

时钟有效的时候，不能让 R 和 S 改变，信号必须在时钟周期的另一半保持稳定。

定义 3.3 主从结构

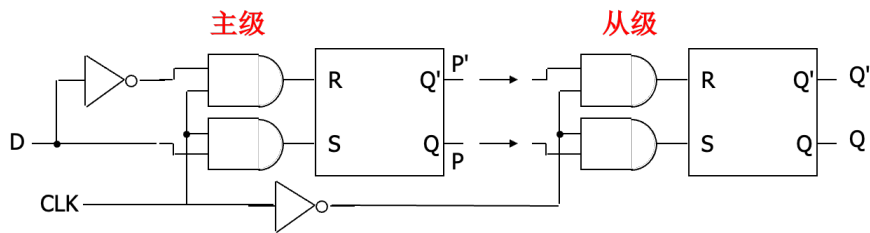
通过交替的时钟断开流：

- 使用正时钟将输入锁存到一个 R-S 锁存器中
- 使用负时钟通过另一个 R-S 锁存器改变输出

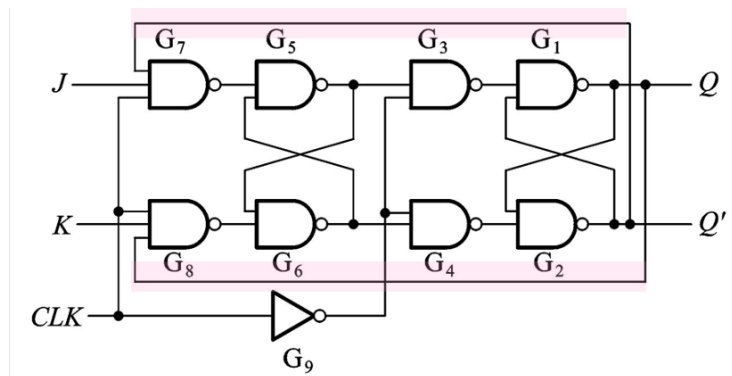


主从结构具有“1 的捕获问题”：在时钟高电平期间，输入端出现一个短暂的“1”脉冲（即使宽度很窄），主锁存器就可能错误地“捕获”这个 1，导致下降沿输出错误的结果。

为了弥补这个问题，我们引入了 D 触发器：



为了解除 $R = S = 1$ 的约束，我们引入了 JK 触发器：



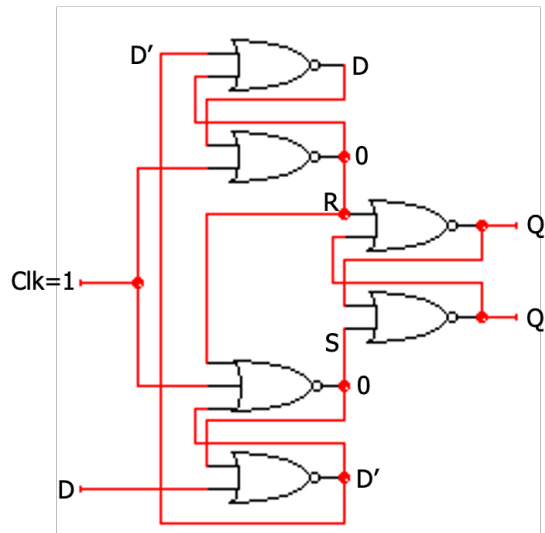
- 若 $J = 1, K = 0, \text{clk} = 1$ 时
 $Q = 1$, 主保持 1; $Q = 0$, 主 = 1; 当 $\text{clk} \downarrow$ 后, 从 = 1
- 若 $J = 0, K = 1, \text{clk} = 1$ 时
 $Q = 1$, 主 = 0; $Q = 0$, 主保持 0; 当 $\text{clk} \downarrow$ 后, 从 = 0
- 若 $J = K = 0, \text{clk} = 1$ 时
 $Q = 1$, 主保持; $Q = 0$, 主保持; 当 $\text{clk} \downarrow$ 后, 从保持
- $J = K = 1, \text{clk} = 1$ 时
 若 $Q = 1$, 则主置 0; 若 $Q = 0$, 则主置 1; 当 $\text{clk} \downarrow$ 后, 从翻转, 即 $Q^* = Q'$



注意：

主从 SR，主为同步 SR， $\text{clk}=1$ 的全部时间里输入信号对主都起控制作用；但主从 JK 在 clk 高电平期间，主只可能翻转一次。

为了彻底解决毛刺的问题，我们引入了边沿触发器：



边沿触发利用了传输门固有的传输延时，制造出一个极小的脉冲采样窗口，当时钟信号中 0 突变到 1 时，输入端的信号被推入锁存结构，随后采样通道被阻塞，毛刺无法影响已经锁存的值。

```

1 module d_ff (q, q_bar, data, clk);
2     input  data, clk;
3     output q, q_bar;
4     reg    q;
5
6     assign q_bar = ~q;
7
8     always @(negedge clk)
9         begin
10            q <= data;
11        end
12 endmodule

```

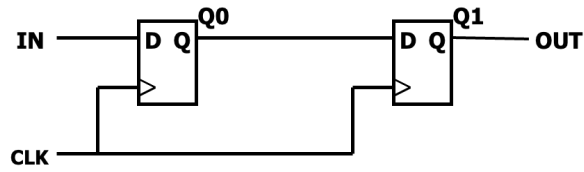
定义 3.4 定时

- 时钟：周期性事件，导致存储单元状态改变，可以是上升沿或下降沿，或高电平或低电平
- 建立时间：在时钟事件之前输入必须稳定的最小时间 (T_{su})
- 保持时间：在时钟事件后输入必须保持稳定的最小时间 (T_h)



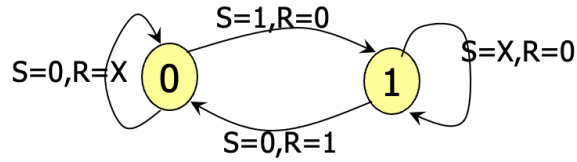
位移寄存器：

- 新值进入第 1 级
- 而第 1 级的存储的值进入第 2 级
- 考虑建立时间、保持时间、传播延迟 (传播延迟必须大于保持时间)

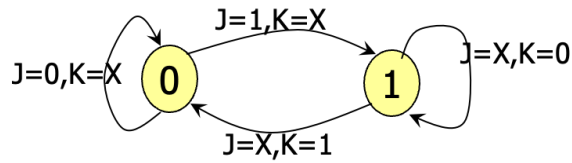


在高性能系统中，由于时钟频率较高，时钟到达触发器的时间与逻辑延迟相当，因此时钟脉冲具有相位差。

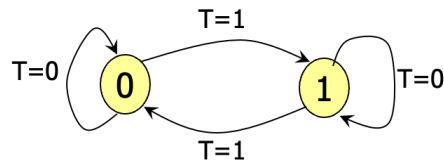
RS 锁存器： $Q^{n+1} = S + R'Q$



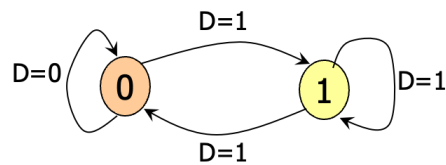
JK 触发器： $Q^{n+1} = JQ^{n'} + K'Q^n$



T 触发器： $Q^{n+1} = TQ^{n'} + T'Q^n$



D 触发器： $Q^{n+1} = D$



注释：

T 触发器就是把 JK 触发器的 JK 两个输入拧成一个 T 输入，如果不嫌麻烦可以实现上述几个触发器和锁存器的相互转换。

§3.2 时序电路

时序电路分为两大类：

- 同步时序电路：所有触发器状态的变化都是在同一个时钟信号下同时发生
- 异步时序电路：触发器状态的变化不是同时发生的

定义 3.5 有限状态机

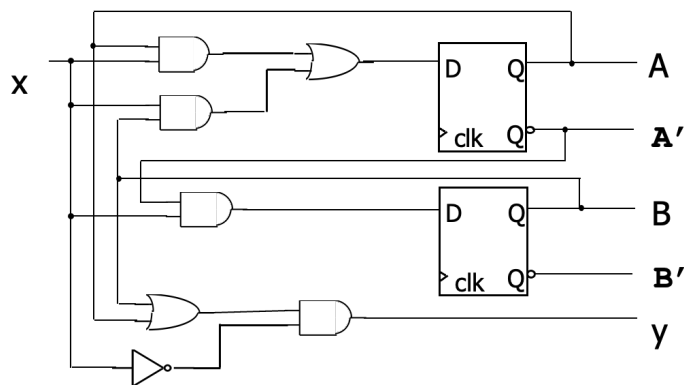
采用输入信号和电路状态的逻辑函数去描述时序电路逻辑功能的方法，分为两大类。

- Mealy 型：输出信号取决于存储电路状态和输入变量
- Moore 型：输出只是存储电路现态的函数，与时钟同步



例题 3.1

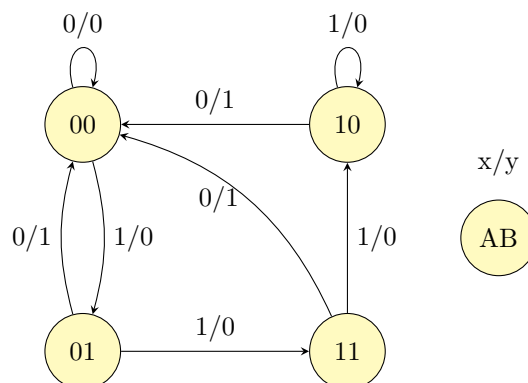
分析下面的时序电路。



解：根据电路逻辑可以列出如下状态转换表。

现态		输入	次态		输出
A	B	x	A*	B*	Y
0	0	0	0	0	0
0	0	1	0	1	0
0	1	0	0	0	1
0	1	1	1	1	0
1	0	0	0	0	1
1	0	1	1	0	0
1	1	0	0	0	1
1	1	1	1	0	0

根据状态表可以画出如下所示的状态转换图。





小结:

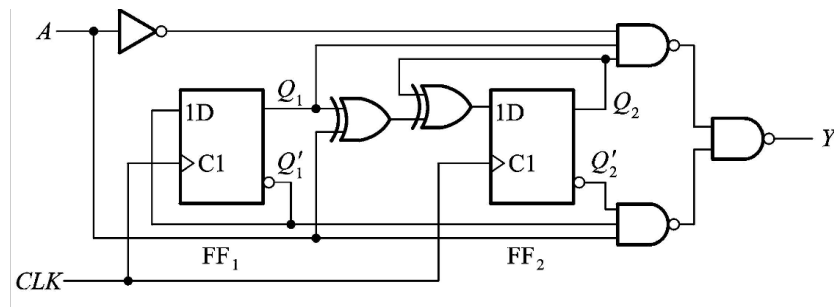
时序电路分析的一般步骤:

- 把触发器的输入方程表示成现态和输入变量的函数
- 将输入方程代入到触发器的特性方程中, 得到状态方程
- 使用对应的状态方程确定状态表中的次态



例题 3.2

分析下面的时序电路。



解:

(1) 驱动方程

$$D_1 = Q_1'$$

$$D_2 = A \oplus Q_1 \oplus Q_2$$

(2) 状态方程

$$Q_1^* = Q_1'$$

$$Q_2^* = A \oplus Q_1 \oplus Q_2$$

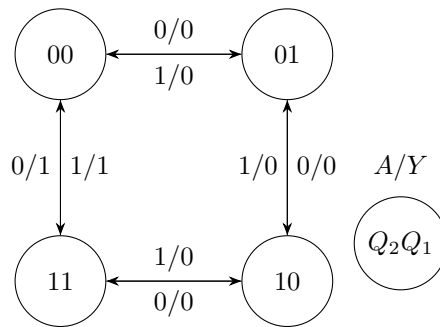
(3) 输出方程

$$Y = A'Q_1Q_2 + AQ_1'Q_2'$$

(4) 状态转换表

A \ Q_2Q_1	00	01	10	11
0	01/0	10/0	11/0	00/1
1	11/1	00/0	01/0	10/0

(5) 有限状态转换图

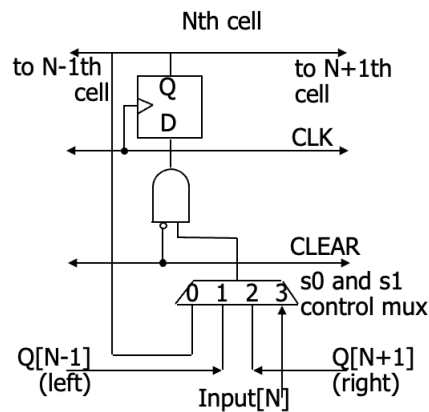


定义 3.6 移位寄存器

- 存储：可以存储 N 位二进制数
- 移位：每一个时钟触发刻，整体数据右移一位或左移一位



一个移位寄存器的基本单元可以用下面的模块表示：



用 Verilog 描述如下：

```

1  module univ_shift (
2      output [3:0] out,    // 4 位并行输出
3      output lo,          // 左移串行输出（最高位）
4      output ro,          // 右移串行输出（最低位）
5      input  [3:0] in,     // 并行加载数据
6      input  li,           // 左移串行输入（填入最低位）
7      input  ri,           // 右移串行输入（填入最高位）
8      input  [1:0] s,      // 模式选择
9      input  clr,          // 异步清零（高有效）
10     input  clk            // 时钟
11 );
12
13     reg [3:0] out;
14     assign lo = out[3];

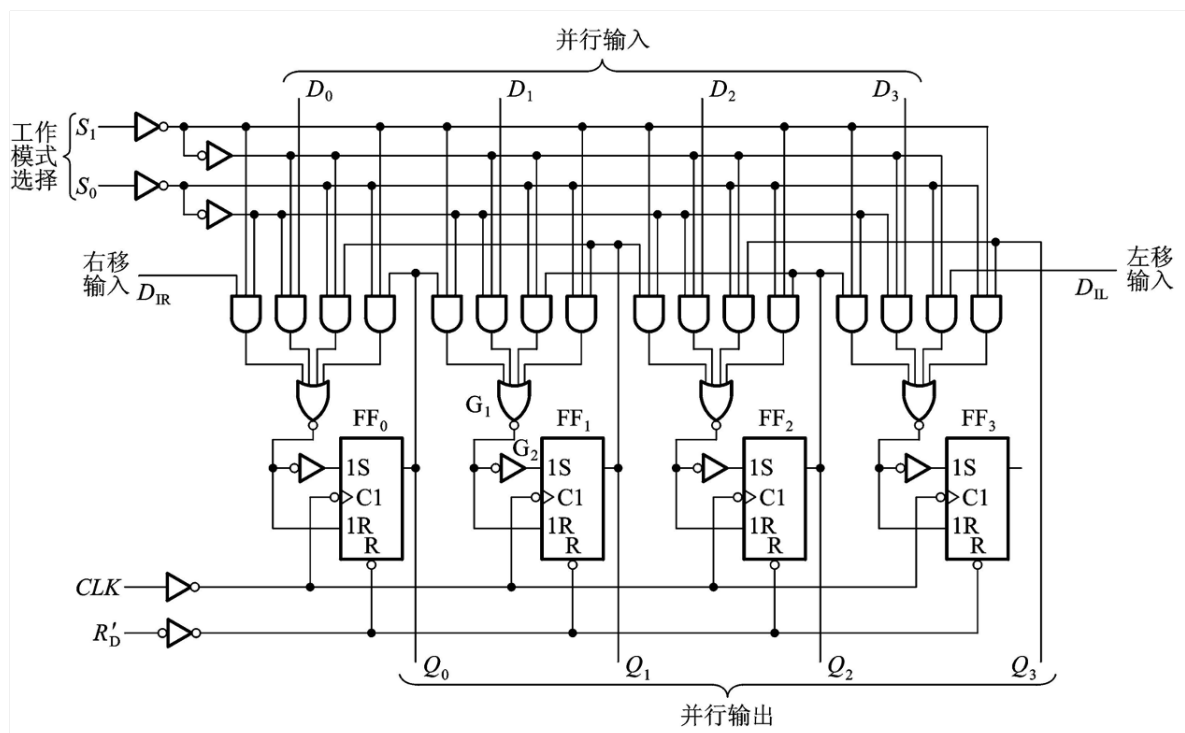
```

```

15     assign ro = out[0];
16
17     always @(posedge clk or posedge clr) begin
18         if (clr)
19             out <= 4'b0;
20         else
21             case (s)
22                 2'b11: out <= in;                // 并行加载
23                 2'b10: out <= {ri, out[3:1]};    // 右移 (MSB为ri, 数据右移)
24                 2'b01: out <= {out[2:0], li};    // 左移 (数据左移, LSB为li)
25                 2'b00: out <= out;              // 保持
26                 default: out <= out;
27             endcase
28     end
29 endmodule

```

4 位移位寄存器的芯片 74LS194A 结构如下:



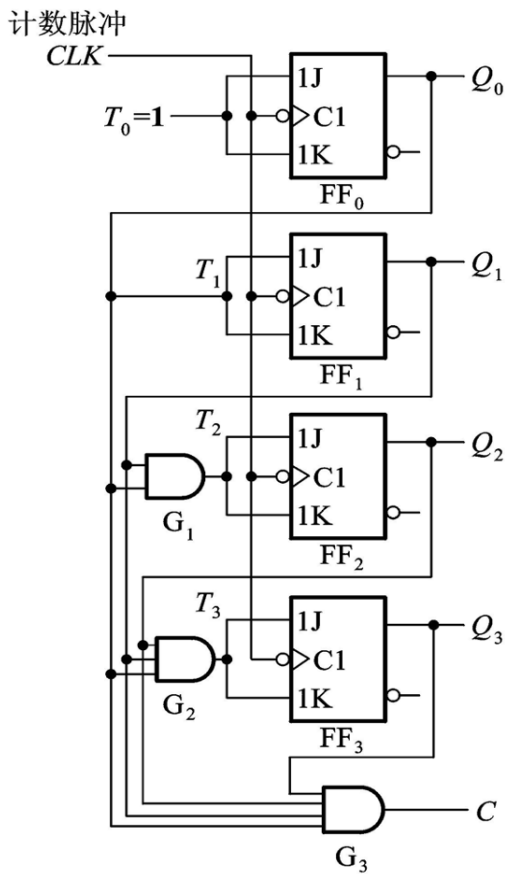
定义 3.7 计数器

用于计数、分频、定时、产生节拍脉冲等的器件。

- 有同步、异步之分
- 可以根据需要设计进制



经典的同步二进制加法器如下:



注释:

利用计数器，我们可以设计出分频器——计数器每升高一位，频率降低二分之一倍。

如果不考虑占空比的要求，可以设计一个 N 位进制计数器，然后将某一状态作为频率输出，即得 N 分频器。

如果对于一个计数器，它的无效状态在经过有限个时钟刻后可以回到循环，则称该电路是可以自启动的。

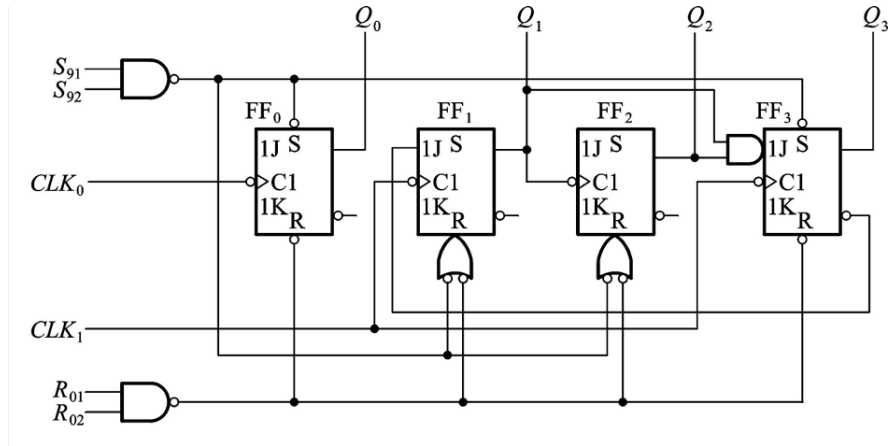
异步计数器和同步计数器不同的是，在末位接受时钟刻更新后，从低位到高位逐位进行进位操作，即高位的时钟刻受低位控制。



注意:

在实际设计中，由于异步计数器容易受到传输延时的影响，后续状态的翻转与设定存在一定的延迟。当工作频率较高时，产生的累计误差可能会产生逻辑错误，时序可能是不收敛的。因此在实际中，通常选用稳定性和分析性更强的同步计数器。

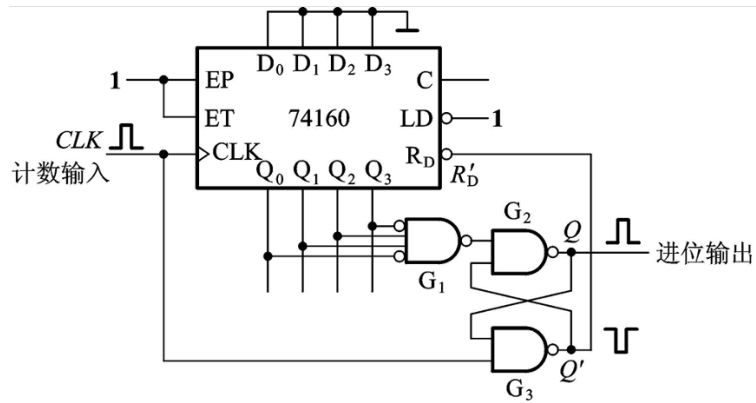
异步十进制计数器 74LS290:



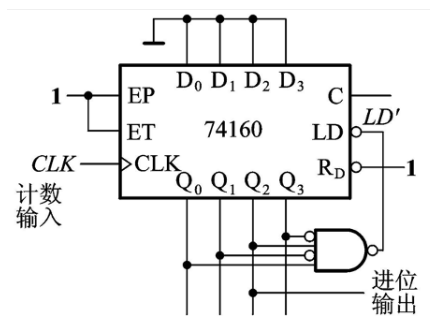
例题 3.3

将十进制 74160 改为六进制计数器。

解：方法 1：修改输出逻辑



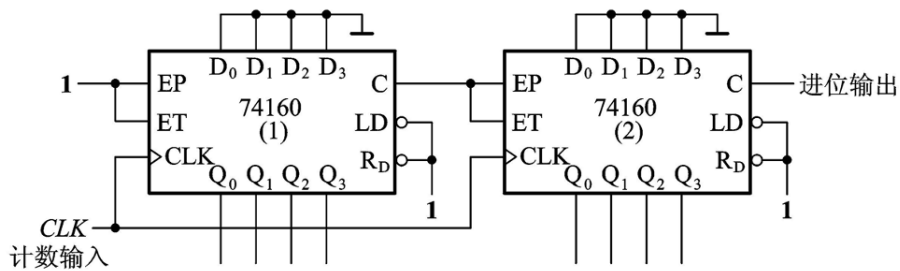
方法 2：预置数



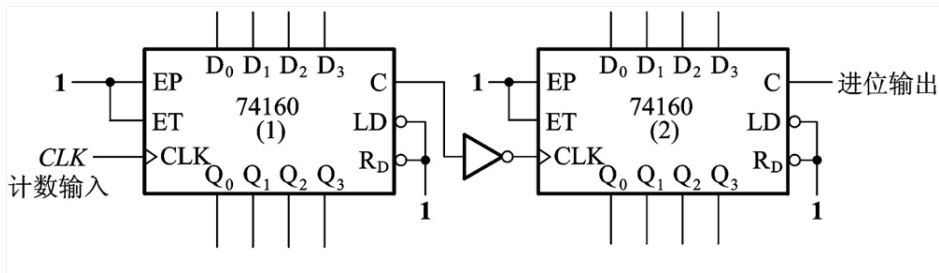
例题 3.4

将两个十进制 74160 改为六进制计数器。

解：方法 1：并行进位法



方法 2：串行进位法

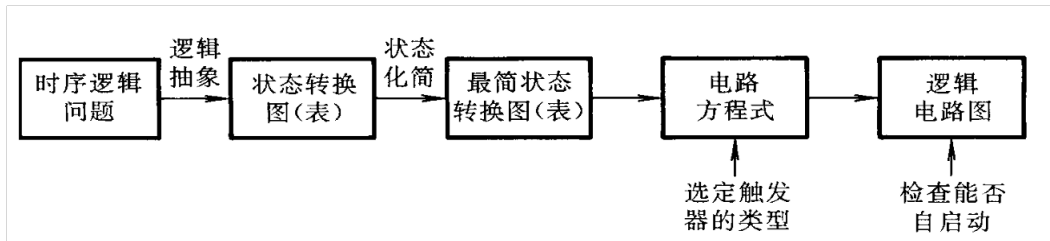


如果要求的计数不可分解（如 29 进制），则可以通过预置数的方法，当电路计数达到预置时清零（或改为预设）。



小结：

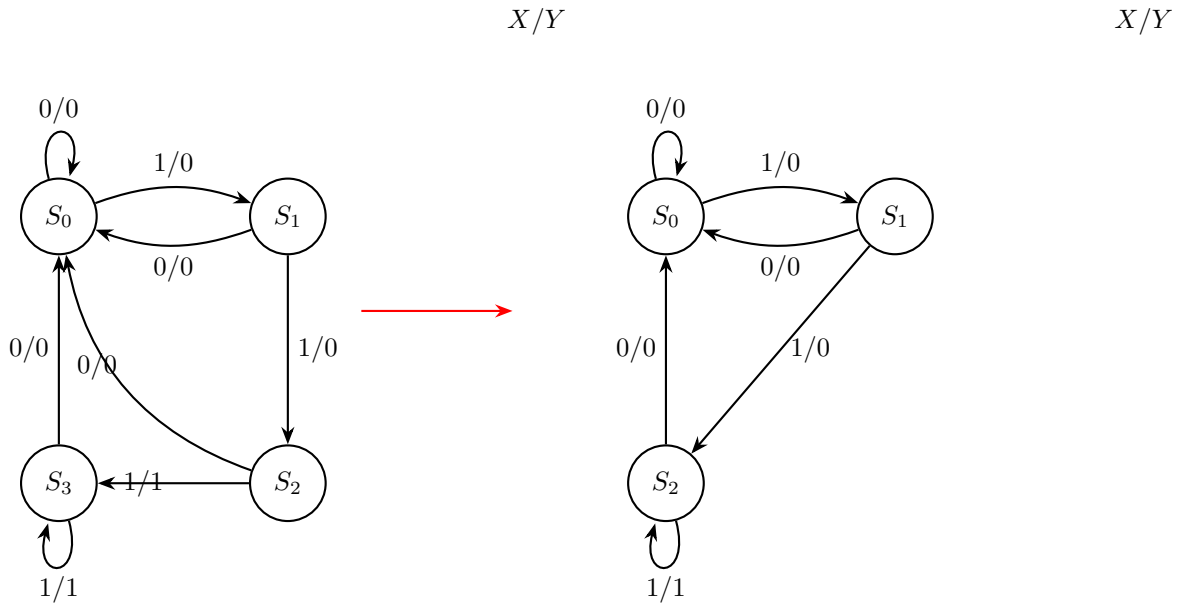
同步时序逻辑电路的设计方法。



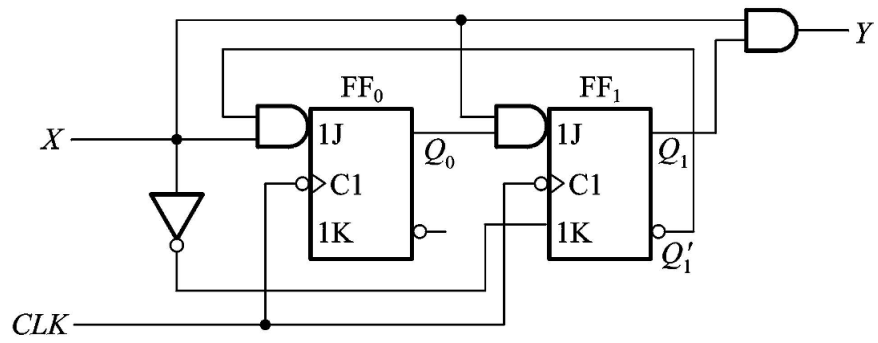
例题 3.5

设计一个串行数据检测器，要求在连续输入三个或三个以上“1”时输出为 1，其余情况下输出为 0。

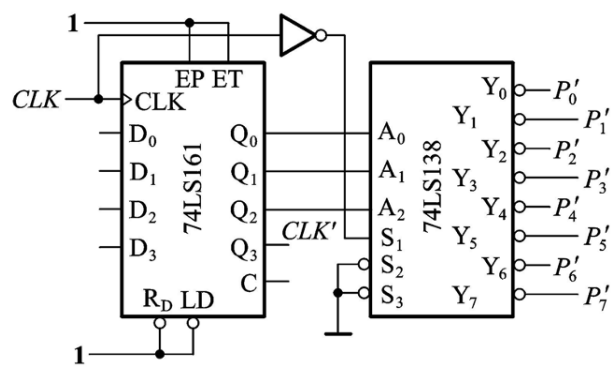
解：根据题目条件，可以列出如下的状态转化图。



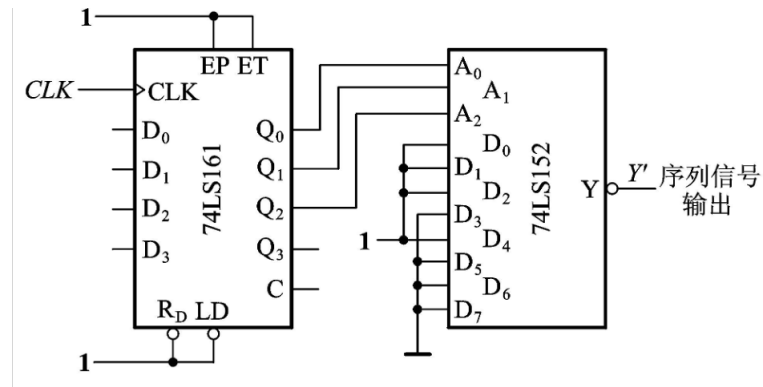
利用卡诺图和状态方程，我们可以用 JK 触发器实现这个电路。



利用计数器和译码器，可以设计顺序节拍脉冲发生器。



利用计数器和数据选择器，可以设计脉冲序列发生器。



小结:

本章主要内容:

- 基本触发器 (RS 锁存器, JK 触发器, T 触发器, D 触发器)
- 基本时序电路 (移位寄存器, 计数器)

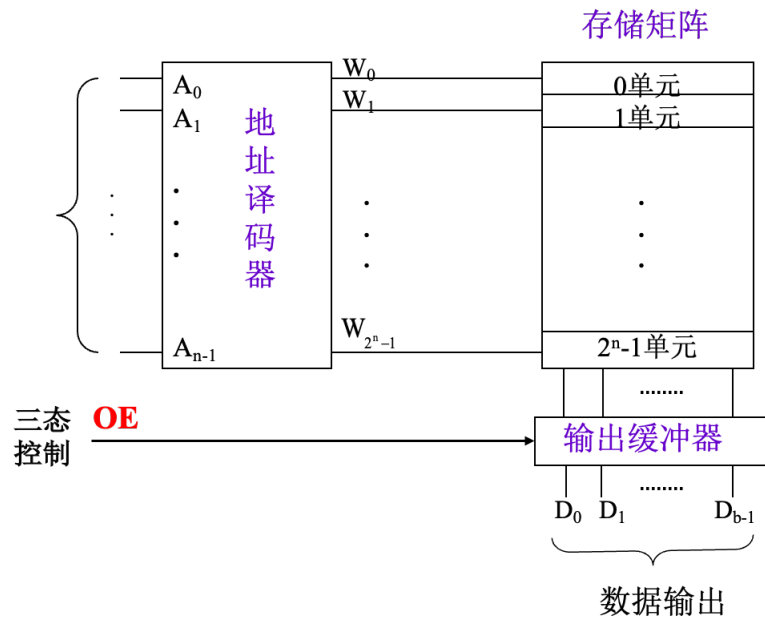
对于一般的时序电路, 都可以从输入输出方程的激励关系进行分析。

4 控制逻辑

§4.1 存储器

定义 4.1 只读存储器 (ROM)

ROM 是各种存储器中结构最简单的一种。在正常工作时它存储的数据是固定不变的，只能读出，不能随时写入



注释:

一般称地址输出线为字线，数据输出线为位线，容量为字 $\times$ 位。

如 15 位地址输入，8 位数据输出，则容量为 $2^{15} \times 8 = 256\text{K}$ 。

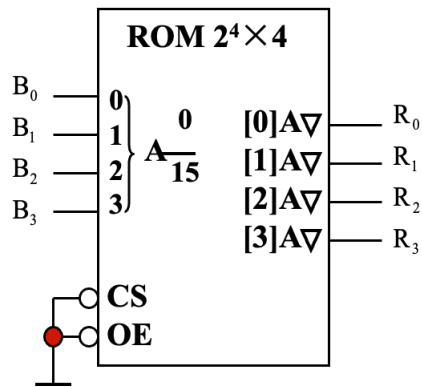
ROM 为组合电路器件：实现组合逻辑函数，实现时序电路中组合逻辑部分。



例题 4.1

用一个 ROM 实现二进制码到格雷码的转换。

解：输入变量为 B_3 、 B_2 、 B_1 、 B_0 ，地址为 4 位；函数 R_3 、 R_2 、 R_1 、 R_0 为 4 个，应选用 $2^4 \times 4$ 的 ROM。



ROM 可以理解为一个查找表（类似 OI 竞赛中的打表法）。

定义 4.2 可编程逻辑阵列 (PLA)

可实现与阵列和或阵列编程的逻辑器件。



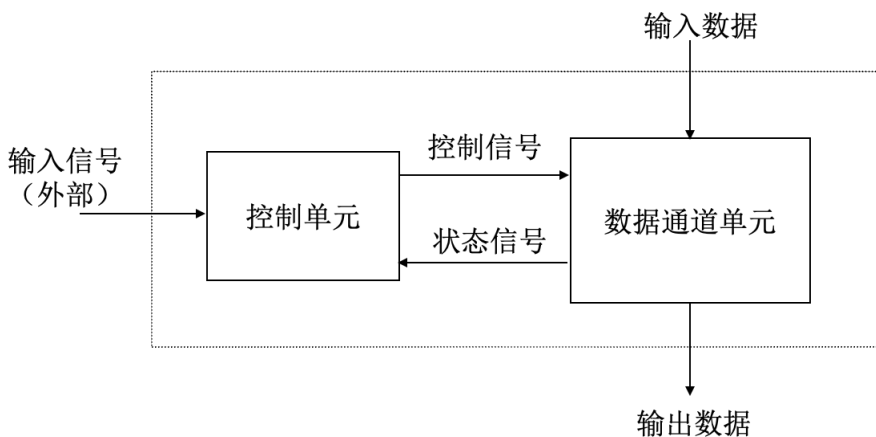
注意：

与 ROM 不同的是，PLA 的与阵列（地址线）也是可编程的，减少了不需要的最小项。

在编程前，所有可能的与阵列和或阵列都是全部连接的，在编程后将不需要的连接熔断（fuse/blow），基本设计方法与组合逻辑相似。

§4.2 控制器

控制单元是为启动数字系统中处理器的微操作提供控制信号时间序列的单元。



定义 4.3 状态机

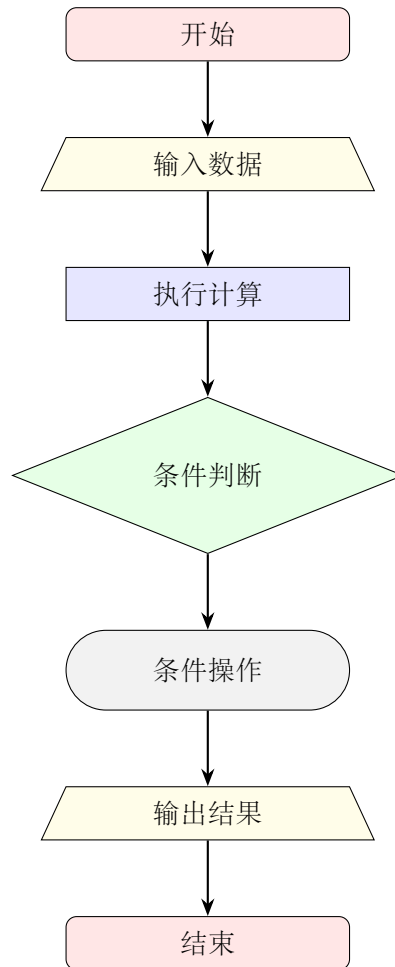
状态机是指按有序方式遍历预先确定的状态序列的数字逻辑功能电路。

状态机有两种基本操作：

- 状态机内部状态转换。遍历某一确定的状态序列，其中次态由次态译码器根据现态和输入条件来确定；
- 根据状态变化（称为状态转移）产生输出信号。输出译码器根据现态和输入条件可确定输出信号。

定义 4.4 算法流程图 (ASM)

ASM 图用来描述控制器不同时间内应完成的一系列操作，指出控制器状态转换、转换条件，以及控制器的输出。



小结：

控制器的几种设计方法：

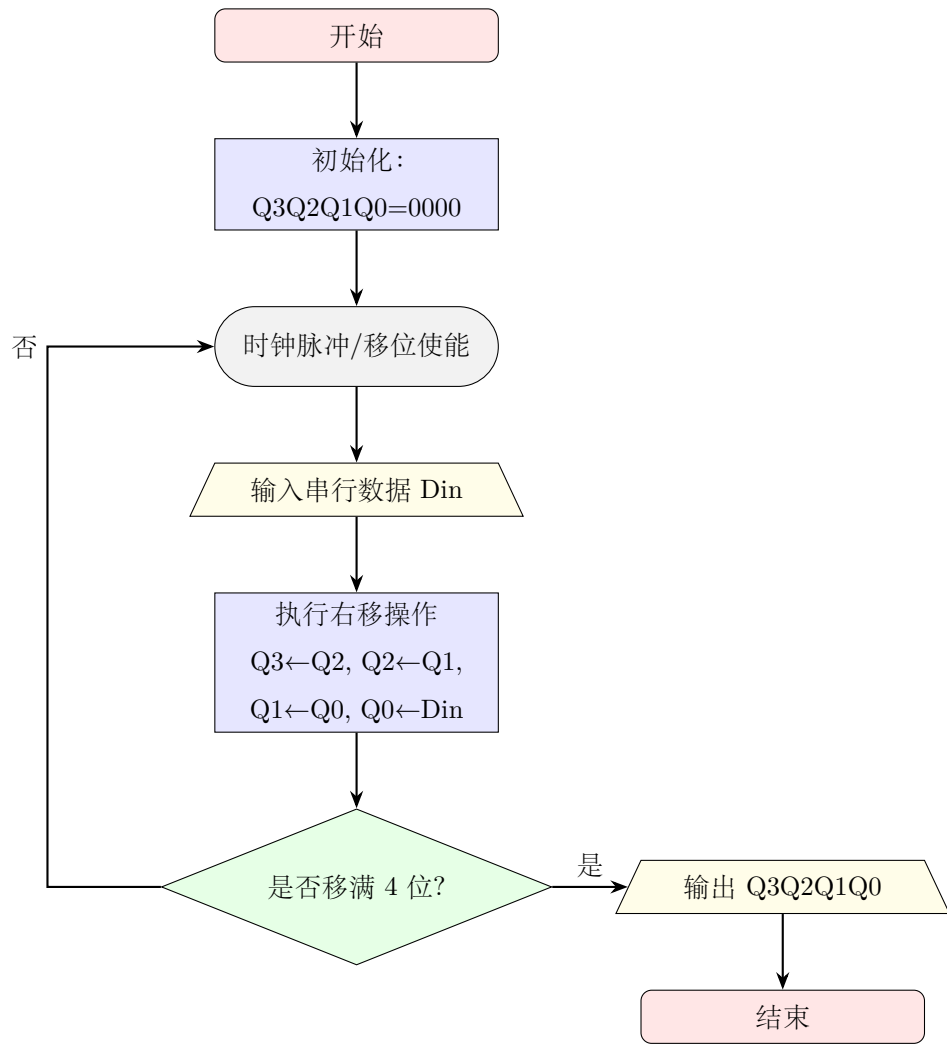
- 每个状态一个触发器 (one-hot, 热位)
- 序列寄存器-译码器法
- 可编程逻辑阵列 PLA 控制法 (选择器法)
- 微程序控制法



例题 4.2

用算法流程图描述四位移位寄存器。

解：描述如下。



注意:

如果工作块不能再一个时钟周期内完成，则必须在 ASM 图中分为几个状态框，在这几个状态之间实现无条件的转移。



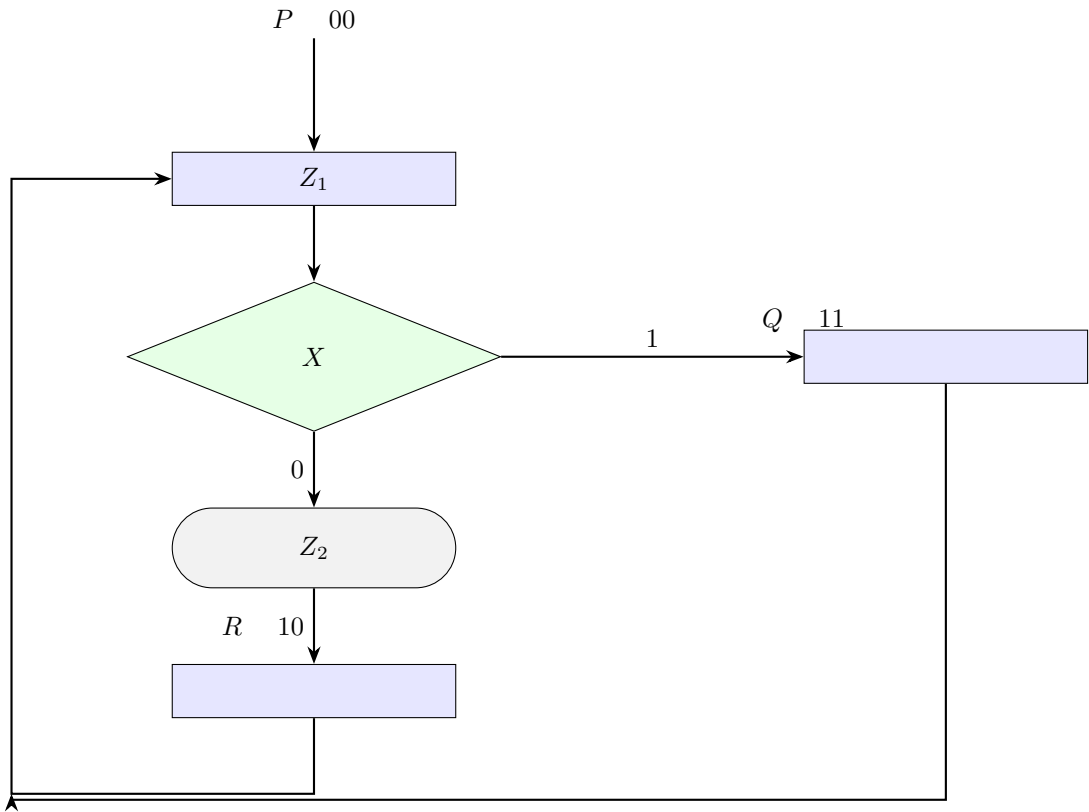
例题 4.3

用数据选择器设计控制器。

解：设计流程如下。

- 决定寄存器次态的数据选择器组成
- 保存现态的寄存器
- 译码器，产生每个控制状态对应的输出

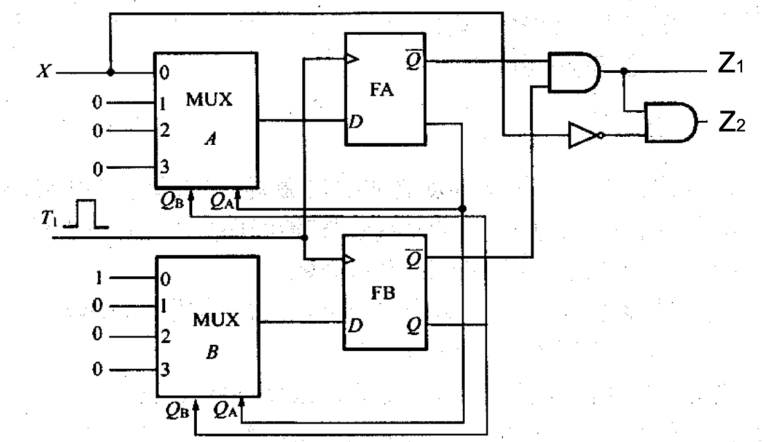
如下图所示的流程图。



转换表如下。

PS(现态)		NS(次态)			转换条件
编码	状态名	状态名	B	A	
0 (00)	P	R	1	0	X'
		Q	1	1	X
2 (10)	R	P	0	0	0
3 (11)	Q	P	0	0	0
1 (01)	-	P	0	0	0

最后设计的电路如下。



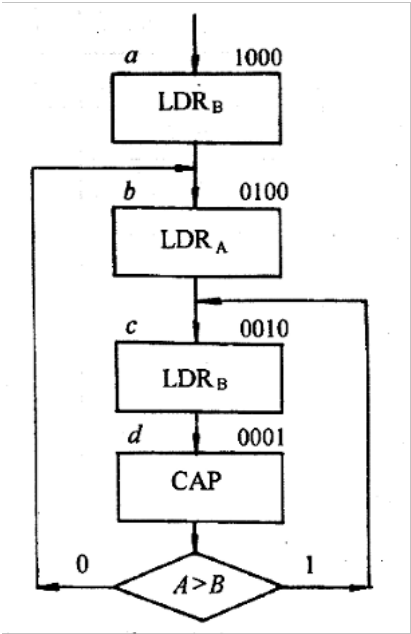
控制器可以大致分为以下几类：

- 计数器型控制器：将所要求的控制状态按一定原则进行编码分配，就可设计出一种状态计数器型控制器；
- 选择器型控制器：多路选择器的输出是触发器的输入，也是触发器的次态激励函数；多路选择器的控制端分别与触发器的 Q 输出端相连；
- 定序型控制器：一位热位（One-hot）设计，产生的电路每一个状态需要一个触发器，每次只有一个触发器为 1，其余触发器都为 0；



例题 4.4

有一个数字比较系统，它能连续对两个二进制数据进行比较，操作过程如下：先将两个数存入寄存器 RA 和 RB，然后进行比较，最后将大数移入寄存器 RA 中。其方框图和 ASM 流程图如图所示。其中 X 为输入信号，LDRA, LDRB 为打入控制信号，CAP 是三态门使能控制信号， $A > B$ 是比较器输出信号。请设计定序型控制器。假设状态变化发生在时序 T1，打入寄存器操作发生在时序 T2，状态周期 $T = T1 + T2$ 。



解：根据流程图可以写出如下的状态转换表：

PS(现态)				NS(次态)				转换条件
Q_a	Q_b	Q_c	Q_d	$Q_a(D)$	$Q_b(D)$	$Q_c(D)$	$Q_d(D)$	C
1	0	0	0	0	1	0	0	初始化置“1” Q_a
0	1	0	0	0	0	1	0	-
0	0	1	0	0	0	0	1	-
0	0	0	1	0	0	1	0	$A > B$
				0	1	0	0	$\overline{A > B}$

次态信号逻辑表达式如下：

$$Q_a(D) = 0$$

$$Q_b(D) = Q_a + \overline{(A > B)} \cdot Q_d$$

$$Q_c(D) = Q_b + (A > B) \cdot Q_d$$

$$Q_d(D) = Q_c$$

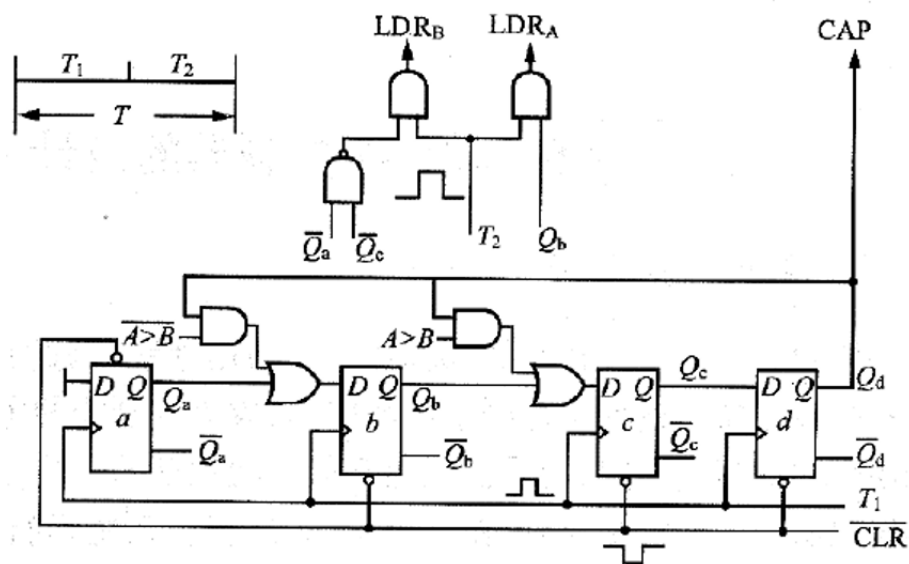
控制信号逻辑表达式如下：

$$LDR_B = (Q_a + Q_c) \cdot T_2$$

$$LDR_A = Q_b \cdot T_2$$

$$CAP = Q_d$$

最终可设计如图所示的电路：



例题 4.5

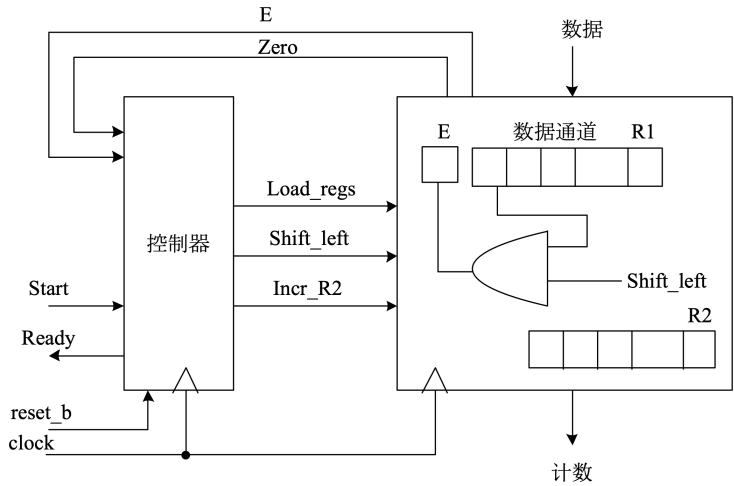
设计一个数字系统来计量寄存器中 1 的个数，画出数字系统电路框图和 ASM 图。

设计要求如下：

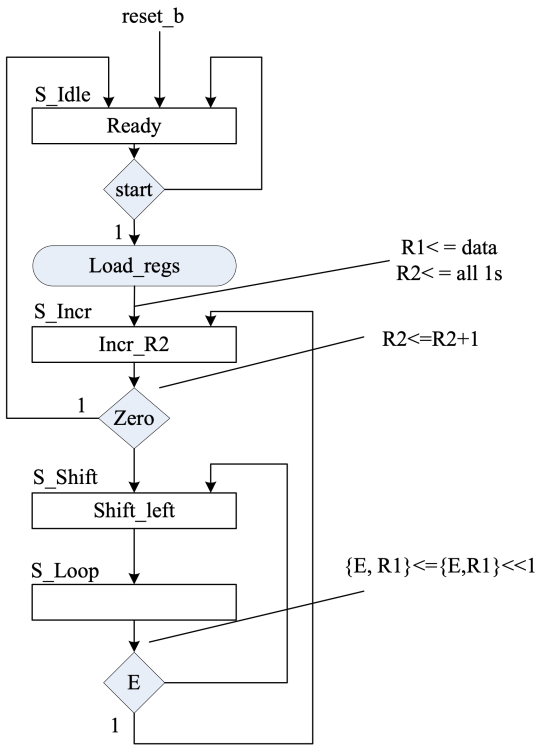
- 设计的数字系统包括两个寄存器 R1 和 R2，以及一个触发器 E。系统对加载到寄存器 R1 中的数字中 1 的个数进行计量。将被计数的数值装载到寄存器 R1，寄存器 R2 等于被计数的 1 的个数，触发器 E 用于逐位处理 R1 中的数据
- 控制器负责控制整个操作流程，包括启动、计数和复位。输入信号 Start 启动系统计数操作，Ready 为控制器的状态指示。控制器采用同步复位，时钟正边沿触发
- 控制器接收从数据路径来的状态输入 E 和 Zero 信号。这两个信号分别指示寄存器中存储的数字字的最高有

效位的内容，以及数据是否为 0 的。E 是触发器输出。Zero 是检测寄存器 R1 是否全 0 的结果输出。当 R1 的值为 0（即 R1 中不包含 1）时，电路输出 Zero = 1

解：电路框图如下：



算法流程图如下：



注释：

考试的时候遇到这种题不要慌，因为大部分人都是交白卷的。



小结:

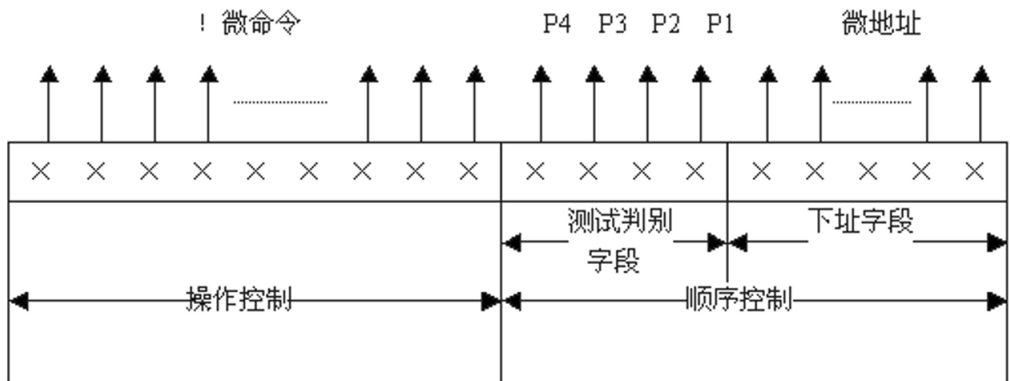
对于常见的数据选择器类型，一般的设计方法如下：

- 决定寄存器次态的数据选择器
- 保存现态的寄存器
- 译码器，产生每个控制状态对应的输出

§4.3 微程序控制

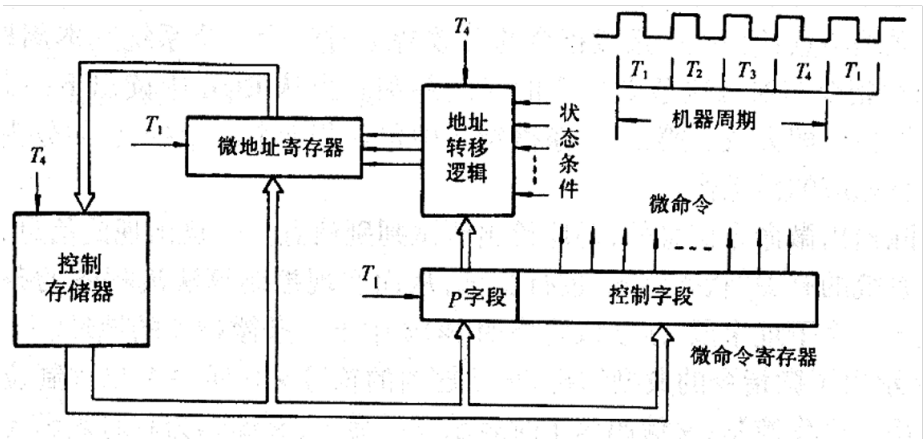
微程序控制的基本思想，就是仿照通常的解题程序的方法，把所有的操作控制信号汇集一起编码成所谓的微指令，存放在一个存储单元里。

- 微命令：控制部件通过控制线向执行部件发出各种控制命令
- 微操作：执行部件接受微命令所执行的操作



微程序控制器的一般结构：

- 控制存储器存放微码程序
- 微命令寄存器存放当前微码指令的控制信息
- 微地址寄存器存放下条微指令的地址
- 地址转移逻辑





例题 4.6

用微处理器实现例 6.4 的数字系统。

解：

当前微地址	微指令二进制代码		
	微命令	判别	下地址
0 0 0 0	1 0 0	0	0 1 0 0
0 1 0 0	0 1 0	0	0 1 0 1
0 1 0 1	1 0 0	0	1 0 0 0
1 0 0 0	0 0 1	1	0 1 0 0



注意：

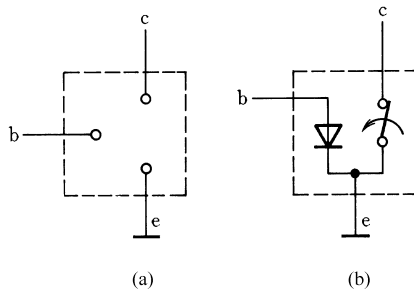
当判别位为 1 的时候，下地址不是由微指令里的下地址（基地址）决定的，而是通过外部硬件电路强制修改。

如果对微控制器感到头疼，请参考《计算机组成》。

5 门电路

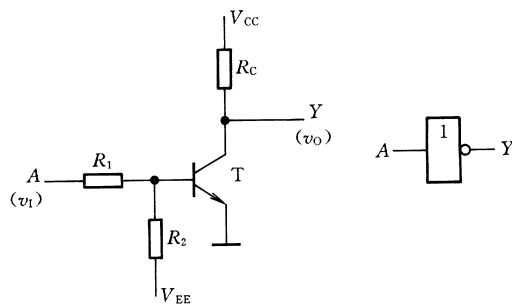
§5.1 TTL

三极管开关电路：



用作开关的三极管处于截止或饱和状态。

三极管非门：

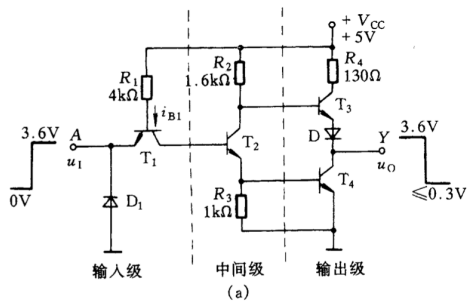


- 接入电阻 R_2 和负电源 V_{EE} ，使三极管可靠地截止
- 当输入信号为高电平时，保证 $I_B > I_{BS}$ ，工作在深饱和状态，三极管输出电平接近于零



例题 5.1

分析下图电路：



解：（定性分析）

- $V_I = V_{IL} = 0.2V$, T1 基极电流 i_{B1} 流入发射极，即由反相器输入端流出，因此 $i_{B2} \approx 0$, T2 截止, T4 也截止. T3 和 D 将导通，输出为高电平 V_{OH}

- $V_I = V_{OH} = 3.6 \text{ V}$, T1 倒置, 即发射极和集电极颠倒; i_{B1} 流入 T2 基极, 使 T2 饱和导通, 从而 T4 饱和导通, T3 和 D 截止, 输出为低电压 V_{OL}

(定量分析)

(1) $u_I = U_{IL} = 0 \text{ V}$ 时

$$u_{B1} = u_{BE1} = 0.7 \text{ V}$$

$$i_{B1} = (V_{CC} - u_{B1})/R_1 = \frac{5 - 0.7}{4} \text{ mA} \approx 1.1 \text{ mA}$$

T₂ 基极不可能供给 T₁ 集电极以电流, 因为在正常情况下稳态时, i_{B2} 是不可能向外流的, 即

$$i_{C1} = 0, \quad I_{BS1} = 0, \quad i_{B1} \gg I_{BS1}$$

因此 T₁ 深度饱和, 故

$$u_{CE1} = U_{CES1} \approx 0 \text{ V}$$

$$u_{B2} = u_I + u_{CE1} = u_I + U_{CES1} \approx 0 \text{ V}$$

所以 T₂、T₄ 截止, T₃、D 导通, T₃ 工作在射极输出状态。由于 T₄ 截止, 只有很小的穿透电 I_{CEO} 流过, 因此 i_{B3} 非常小, 故

$$u_{B3} = V_{CC} - i_{B3}R_2 \approx V_{CC} = 5 \text{ V}$$

$$u_O = u_{B3} - u_{BE3} - u_D = (5 - 0.7 - 0.7) \text{ V} = 3.6 \text{ V}$$

输出为高电平。

(2) $u_I = U_{IH} = 3.6 \text{ V}$ 时

若假定 T₁ 发射结仍正向导通, 则

$$u_{B1} = u_I + u_{BE1} = (3.6 + 0.7) \text{ V} = 4.3 \text{ V}$$

看一看 T₁ 集电结、T₂ 和 T₄ 发射结这三个 PN 结的情况, 若 $u_{B1} = 4.3 \text{ V}$, 即从 T₁ 基极到地有 4.3 V 压降, 那么三个 PN 结, 每一个上面将会有 1.3 V 多一点的正向压降, 所以它们必然都导通, 从而使

$$u_{B1} = u_{BC1} + u_{BE2} + u_{BE4} = (0.7 + 0.7 + 0.7) \text{ V} = 2.1 \text{ V}$$

即 u_{B1} 被钳位在 2.1 V, T₁ 发射结正向导通的假定不成立。此时, 由于

$$u_{E1} = 3.6 \text{ V}, \quad u_{B1} = 2.1 \text{ V}, \quad u_{C1} = u_{BE2} + u_{BE4} = (0.7 + 0.7) \text{ V} = 1.4 \text{ V}$$

使 T₁ 发射结反向偏置、集电结正向偏置, 工作在倒置状态——集电极和发射极颠倒使用。T₁ 倒置时, 电流放大系数 β_i 很少, 在 0.02 左右。因此 T₁、T₂ 基极电流

$$i_{B1} = (V_{CC} - u_{B1})/R_1 = \frac{5 - 2.1}{4} \text{ mA} \approx 0.725 \text{ mA}$$

$$i_{B2} = i_{B1} + \beta_i i_{B1} = (0.725 + 0.02 \times 0.725) \text{ mA} \approx 0.74 \text{ mA}$$

假设 T_2 饱和导通，则

$$u_{C2} = u_{CES2} + u_{BE4} = (0.3 + 0.7) \text{ V} = 1 \text{ V}$$

$u_{B3} = u_{C2} = 1 \text{ V}$ ，即 T_3 基极对地的电压只有 1 V，显然 T_3 、D 将截止。故

$$I_{CS2} = (V_{CC} - u_{C2})/R_2 = \frac{5 - 1}{1.6} \text{ mA} = 2.5 \text{ mA}$$

$$I_{BS2} = I_{CS2}/\beta_2 = \frac{2.5}{20} \text{ mA} = 0.125 \text{ mA}$$

因为 $i_{B2} > I_{BS2}$ ，所以 T_2 饱和的假设成立， T_3 、D 截止的推论也是正确的。 T_4 的工作情况：

$$i_{B4} = i_{E2} - i_{R3}$$

$$i_{E2} = i_{B2} + I_{CS2} = (0.74 + 2.5) \text{ mA} = 3.24 \text{ mA}$$

$$i_{R3} = u_{E2}/R_3 = u_{BE4}/R_3 = \frac{0.7}{1} \text{ mA} = 0.7 \text{ mA}$$

$$i_{B4} = i_{E2} - i_{R3} = (3.24 - 0.7) \text{ mA} = 2.54 \text{ mA}$$

由于 T_3 、D 是截止的，故

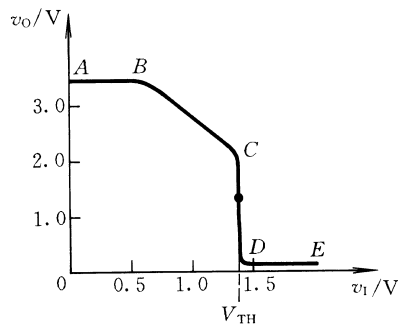
$$I_{CS4} = 0, \quad I_{BS4} = 0$$

$$i_{B4} \gg I_{BS4}$$

所以 T_4 深度饱和， u_O 为低电平：

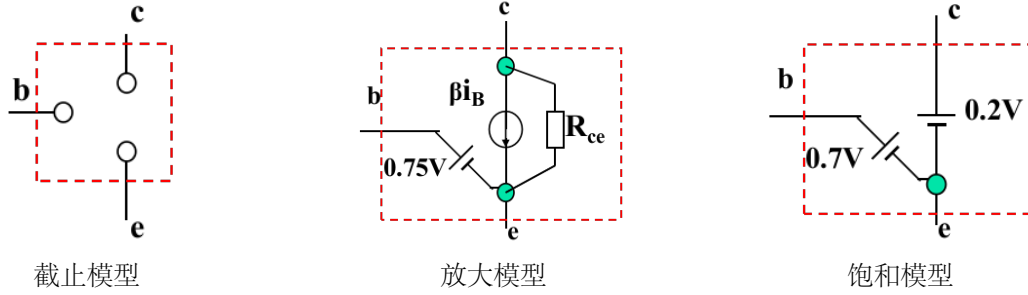
$$u_O = U_{CES4} \leq 0.3 \text{ V}$$

TTL 反相器的电压传输特性：

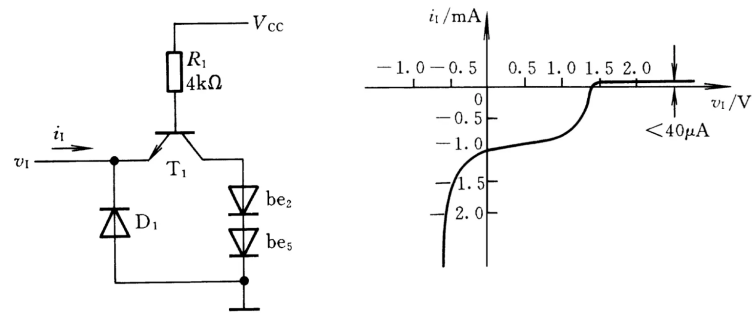


注释：

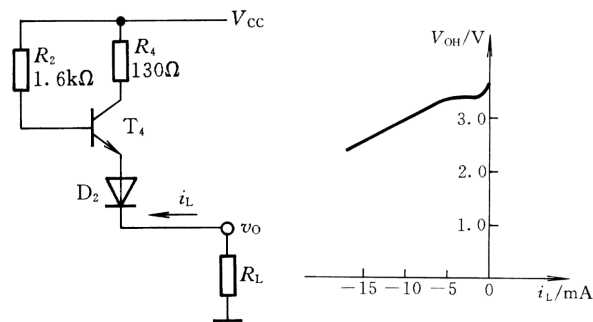
AB 段称为截止区，BC 段称为线性段，CD 称为转折段，DE 称为饱和区。
下面是各区域的等效电路模型。



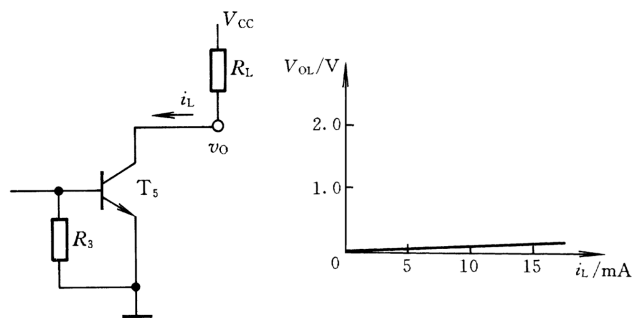
TTL 反相器的输入特性:



TTL 反相器的输出特性:

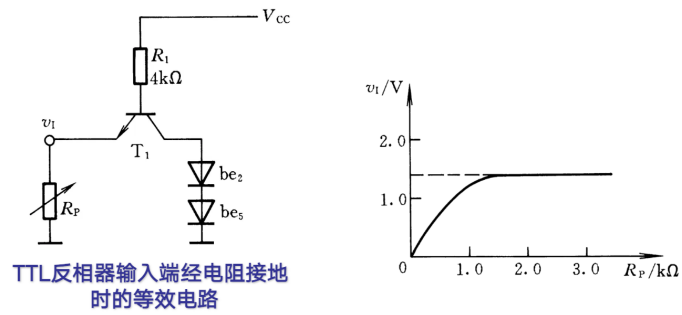


高电平等效电路和高电平输出特性



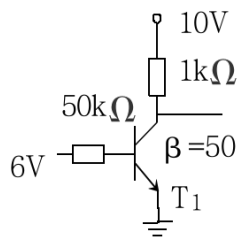
低电平等效电路和低电平输出特性

TTL 输入端负载特性:



例题 5.2

判断三极管 T 的状态



解:

$$i_B \approx \frac{6\text{ V} - 0.7\text{ V}}{50\text{ k}\Omega} = 0.106\text{ mA}$$

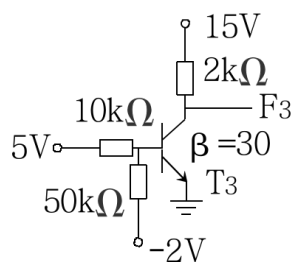
$$I_{BS} \approx \frac{10\text{ V}}{50 \times 1\text{ k}\Omega} = 0.2\text{ mA}$$

$\therefore i_b < I_{BS} \quad \therefore$ 放大状态



例题 5.3

判断三极管 T 的状态



解:

$$i_B \approx \frac{5\text{ V} - 0.7\text{ V}}{10\text{ k}\Omega} - \frac{0.7\text{ V} - (-2\text{ V})}{50\text{ k}\Omega} = 0.376\text{ mA}$$

$$I_{BS} \approx \frac{15\text{ V}}{30 \times 2\text{ k}\Omega} = 0.25\text{ mA}$$

$\therefore i_b > I_{BS} \quad \therefore$ 饱和状态



注意:

首先需要判断三极管是否导通，再进行放大和饱和的判断。



小结:

三极管工作状态的判断:

方法 1: 求临界饱和基极电流 I_{BS}

$$I_{BS} = \frac{V_{CC} - V_{CE(sat)}}{\beta \cdot (R_C + R_{CE})} \approx \frac{V_{CC}}{\beta \cdot R_C}$$

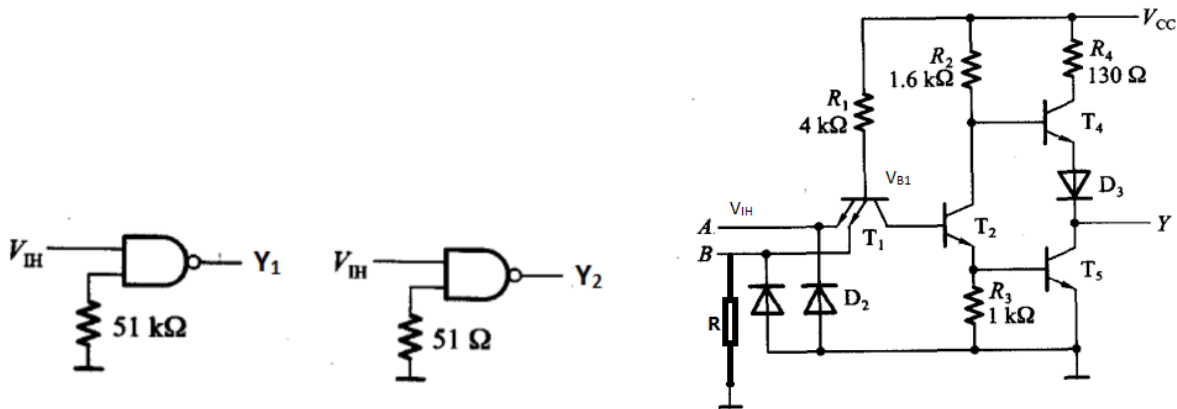
- $i_B > I_{BS}$: 饱和
- $i_B < I_{BS}$: 放大

方法 2: 假设放大, 验证是否正确? (计算 V_{CE} 值: $V_{CE} > 0.7\text{ V}$, 放大, 否则饱和)



例题 5.4

指出下图中两个门电路的输出是什么状态 (高电平、低电平或高阻态)。已知这些门电路都是 74 系列 TTL 电路。



解:

右上图的两个与非门, 一端都是接高电平, 另一端其中一个是通过 $51\text{ k}\Omega$ 电阻接地, 另一个是通过 51Ω 电阻接地。输出状态是 Y_1 为低电平, Y_2 为高电平。具体分析如下:

右下图是 TTL 与非门电路, 当 A、B 两个输入端的电压都大于 1.4 V 时, T_1 的基极电压 V_{B1} 大于 2.1 V , T_2 导通, T_5 导通, T_4 截止, Y 为低电平; 当 A、B 其中有一个小于 1.4 V 时, V_{B1} 小于 2.1 V , T_5 截止, T_4 导通, Y 为高电平。

A 端输入 V_{IH} , 为高电平, 所以与非门输出取决于 B 端输入电平, 即电阻 R 两端的电压。

当电阻 R 两端的电压为临界值 1.4 V 时, 可以按如图所示的电流通路列出等式:

$$\frac{V_{CC} - 0.7}{R_1 + R} \cdot R = 1.4\text{ V}$$

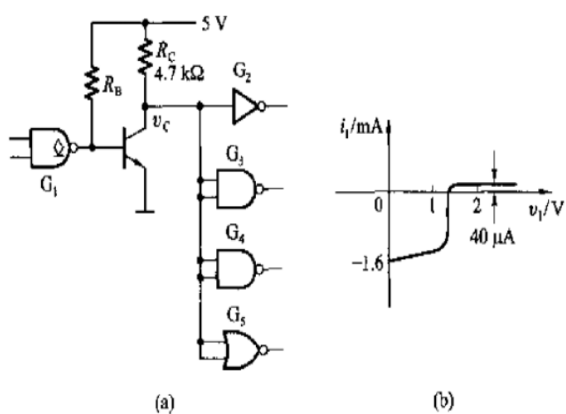
设 $V_{CC} = 5V$ ，求得 R 值为 $1.9k\Omega$ 。

所以，当 B 端接地电阻大于 $1.9k\Omega$ 时，B 端电压大于 $1.4V$ ，Y 为低电平；当 B 端电阻小于 $1.9k\Omega$ 时，B 端电压小于 $1.4V$ ，Y 为高电平。



例题 5.5

图 (a) 所示电路中，已知三极管导通时 $V_{BE} = 0.7V$ ，饱和压降 $V_{CE(sat)} = 0.3V$ ，饱和导通内阻为 $R_{CE(sat)} = 20\Omega$ ，三极管的电流放大系数 $\beta = 100$ 。OC 门 G_1 输出管截止时的漏电流约为 $50\mu A$ ，导通时允许的最大负载电流为 $16mA$ ，输出低电平 $\leq 0.3V$ 。 $G_2 \sim G_5$ 均为 74 系列 TTL 电路，其中 G_2 为反相器， G_3 和 G_4 是与非门， G_5 是或非门，它们的输入特性如图 (b) 所示。试问：在三极管集电极输出的高、低电压满足 $V_{OH} \geq 3.5V$ 、 $V_{OL} \leq 0.3V$ 的条件下， R_B 的取值范围有多大？



解：

首先，根据三极管饱和导通时的要求可求得 R_B 的最大允许值。三极管的临界饱和基极电流应为

$$I_{BS} = \frac{1}{\beta} \left(\frac{V_{CC} - V_{CES}}{R_C + R_{CE(sat)}} + 5I_{IL} \right) = \frac{1}{100} \left(\frac{5 - 0.3}{4.7} + 5 \times 1.6 \right) mA = 0.09mA$$

因此，可得流经 R_B 的电流应为

$$\frac{V_{CC} - V_{BE}}{R_B} = 0.09 + 0.05 = 0.14mA$$

于是，

$$R_B = \frac{V_{CC} - V_{BE}}{0.14} = \frac{4.3}{0.14} k\Omega = 30.7k\Omega$$

又根据 OC 门导通时允许的最大负载电流为 $16mA$ ，可求得 R_B 的最小允许值。

$$R_B = \frac{V_{CC} - V_{OL}}{16} = \frac{4.7}{16} k\Omega = 0.29k\Omega$$

故 R_B 的取值范围是 $0.29k\Omega < R_B < 30.7k\Omega$

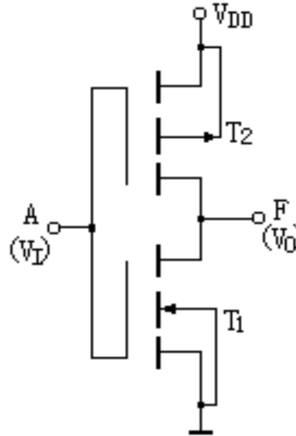


注意:

由于 TTL 门电路的结构, 当低电平接到多个与非门上时, 流经的电流是与非门的数量 ($m = n$), 而其余情况 (高电平接与非门、低电平/高电平接或非门) 时均为端口数 ($m = 2n$)。

§5.2 CMOS

CMOS 反相器:



注释:

CMOS 由 NMOS 和 PMOS 互相连接而成, 栅极相连做输入端, 漏极相连做输出端。衬底与漏源间的 PN 结始终处于反偏, NMOS 管的衬底总是接到电路的最低电位, PMOS 管的衬底总是接到电路的最高电位。

工作原理:

1. 输入为低电平 $V_{IL} = 0V$ 时

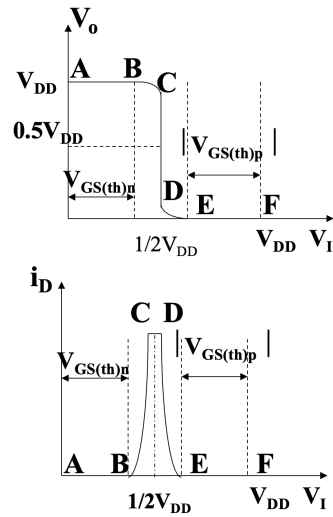
$$V_{GS1} < V_{GS(th)N} \rightarrow T_1 \text{管截止}$$

$$|V_{GS2}| > |V_{GS(th)P}| \rightarrow T_2 \text{导通}$$

电路中电流近似为零 (忽略 T_1 的截止漏电流), V_{DD} 主要降落在 T_1 上, 输出为高电平 $V_{OH} \approx V_{DD}$

2. 输入为高电平 $V_{IH} = V_{DD}$ 时, T_1 通 T_2 止, V_{DD} 主要降在 T_2 上, 输出为低电平 $V_{OL} \approx 0V$

CMOS 传输特性:



CMOS 在稳态功耗几乎为 0，因此只需计算翻转状态时的功耗。



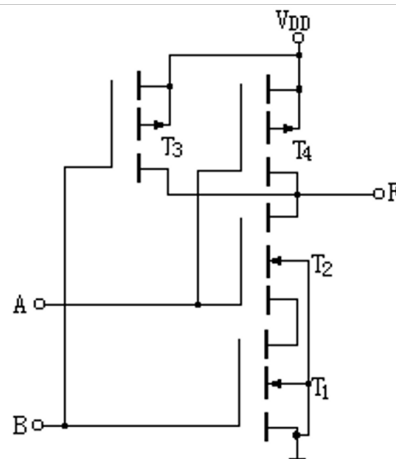
注意：

CMOS 具有传输延时问题，主要原因如下。

- MOS 管开关延时
- 分布参数
- 负载电容

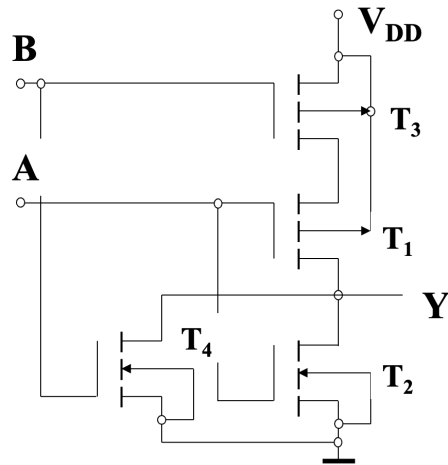
$$t_{pd} = \frac{t_{pHL} + t_{pLH}}{2}$$

CMOS 与非门：

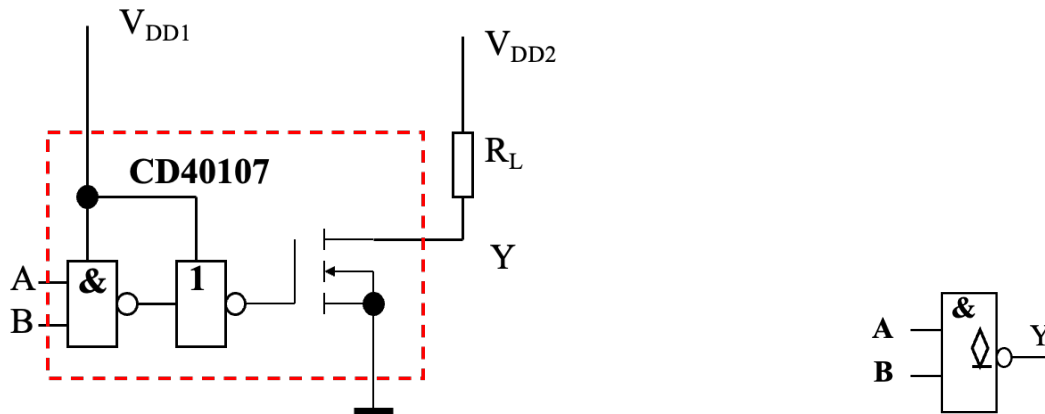


CMOS“与非”门

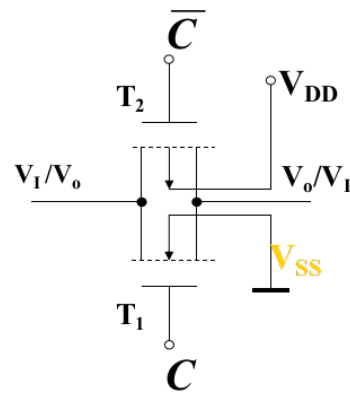
CMOS 或非门：



漏极开路门电路：



CMOS 传输门：



当 $C = 0, \bar{C} = 1$ 时

$$V_{GSN} \leq 0, V_{GSP} \geq 0$$

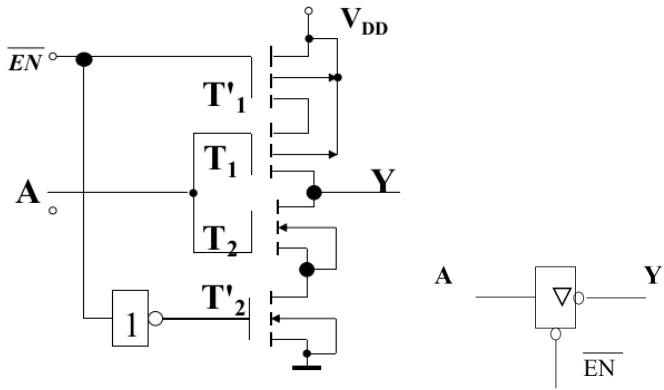
T_1 、 T_2 都截止



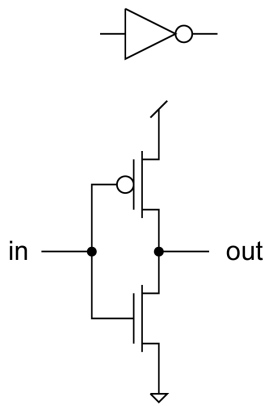
注释:

传输门相当于一个理想的开关，且是一个双向开关，可传输模拟信号。
 $C = 0$ ，开关断开； $C = 1$ ，开关接通。

CMOS 三态门:



$\overline{EN} = 0$, T'_1 , T'_2 导通, $Y = \overline{A}$
 $\overline{EN} = 1$, T'_1 , T'_2 截止, Y 为高阻态
 CMOS 非门:



注释:

国际期刊上常用圆圈表示 PMOS，三角表示 NMOS。

p 网络和 n 网络的逻辑关系式是对偶的，因此只用分析上拉或下拉的逻辑表达式即可。



小结:

一些简单的 MOSFET 连线规则:

- nFET 仅用来传导逻辑 0
- pFET 仅用来传导逻辑 1
- 如与非门



注意:

CMOS 门电路未使用输入端口的规定:

- CMOS 电路的输入端口不能悬空, 所有输入端口要么连接到固定电压位 (地电压 GND 或电源电压 V_{DD}), 要么连接到其它输入信号。
- 未连接的 CMOS 输入, 容易受到噪声和静电荷的影响, 这些噪声和静电电荷很容易使 P 沟道和 N 沟道 MOS-FET 偏置在导电状态, 从而导致电路功耗增加, 及电路过热。

由于上课内容与作业内容严重不符, 下面给出我查找到的判断输入电位的方法。



小结:

TTL 和 CMOS 逻辑门输入的逻辑判断

1. 对于 TTL 来说, 输入的逻辑与接地电阻有关
 - 高电平、 V_{CC} 代表逻辑 1
 - 小电阻接地、小电阻接低电平代表逻辑 0
 - 大电阻接地、大电阻接低电平代表逻辑 1
 - 悬空代表逻辑 1
 - 对于 OC 门来说, 如果内部逻辑运算为 1, 则三极管截止, 对外表现为高阻态, 需要上拉电阻接到 V_{CC} 体现逻辑 1
 - 对于三态门来说, 如果控制端接电源, 则逻辑正常运算, 否则断开, 表现为高阻态
2. 对于 CMOS 来说, 输入的逻辑与接地电阻无关
 - 无论接地/低电平的大小如何, 逻辑始终为 0
 - CMOS 输入禁止悬空
 - OD 门与 OC 门的运算类似

6 脉冲电路

§6.1 触发器电路

脉冲电路有如下基本参数：

- 脉冲周期 T
- 脉冲幅度 V_m
- 脉冲宽度 T_w
- 上升时间 t_r
- 下降时间 t_f
- 占空比 $q = \frac{T_w}{T}$



注释：

本课程所有脉冲电路都可化简为一阶电路，可以用三要素法求解：

$$\begin{cases} \text{时间常数}\tau: & \tau = RC, \tau = \frac{L}{R} \\ \text{初始值}x(0^+): & v_c(0^+) = v_c(0^-), i_L(0^+) = i_L(0^-) \\ \text{稳态值}x(\infty): & v(\infty), i(\infty) \end{cases}$$

由三要素可以确定电路的响应：

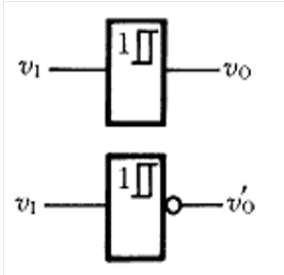
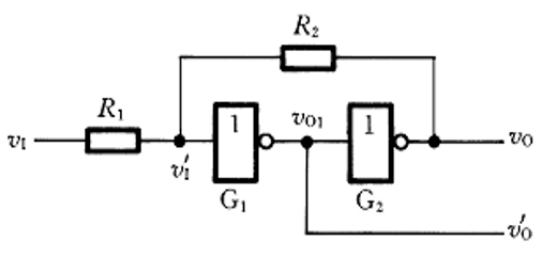
$$x(t) = x(\infty) + [x(0^+) - x(\infty)] e^{-t/\tau}$$

微分方程法和积分变换法可以参考《电子电路基础》或《电路原理》课。

定义 6.1 施密特触发器

输入信号从低电平上升的过程中，电路状态转换时对应的输入电平，与输入信号从高电平下降过程中对应的输入转换电平不同。

- 在输入信号从低电平上升到高电平的过程中使电路状态发生变化的输入电压称为正向阈值电压 V_{T+}
- 在输入信号从高电平下降到低电平的过程中使电路状态发生变化的输入电压称为负向阈值电压 V_{T-}





注意：

施密特触发器要求 $R_1 < R_2$ ，因为

$$V_{T+} = \left(1 + \frac{R_1}{R_2}\right) \cdot V_{TH} \quad V_{T-} = \left(1 - \frac{R_1}{R_2}\right) \cdot V_{TH}$$

其中 $V_{TH} = \frac{1}{2}V_{DD}$ ， $R_1 < R_2$ 保证了阈值电压不会超过电源电压，也不会低于 0。

对于由 RC 组成的微积分电路：

- 积分电路的输出取自于电容，一般要求时间常数小于输入波形宽度的 $\frac{1}{10}$
- 微分电路的输出取自于电阻，一般要求时间常数大于输入波形宽度的 10 倍

定义 6.2 单稳态触发器

单稳态触发器只有一个稳态，要么是 0，要么是 1，剩下的一个状态称为暂稳态。

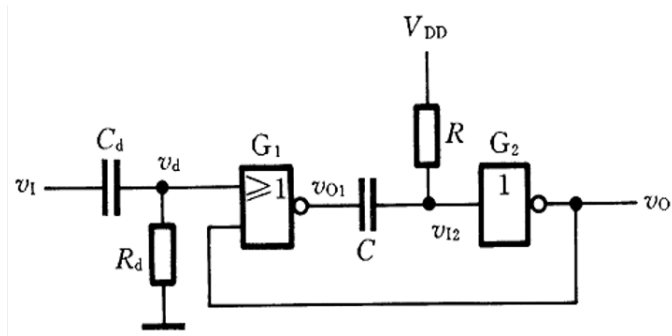
在外部脉冲的作用下，单稳态触发器可以进入暂稳态，在暂稳态持续一段时间后自动返回稳态。



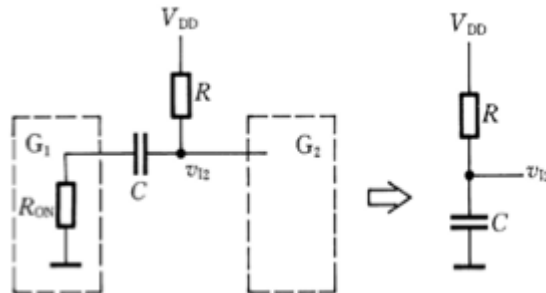
注意：

暂稳态持续时间与电路参数有关，与触发脉冲的宽度和幅度无关。

对于微分型单稳态触发器：

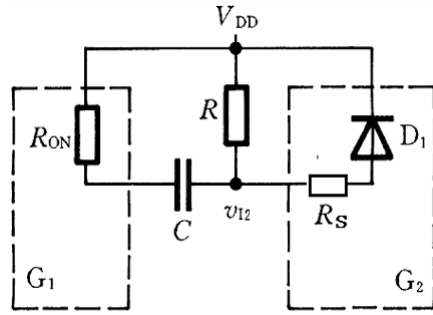


其稳态为输出为 0，当外界输入正脉冲时，电路触发正反馈，迅速拉高输出电平，进入暂稳态：



理想状况下，触发电平为 $\frac{1}{2}V_{DD}$ ，故暂稳态的持续时间为 $t_w = 0.69RC$ 。

从暂稳态回到稳态需要进行放电过程，称为恢复期：



理论上来说恢复到稳态需要需要无穷长的时间，在实际应用中一般在 3~5 个时间常数后即可认为达到稳态。

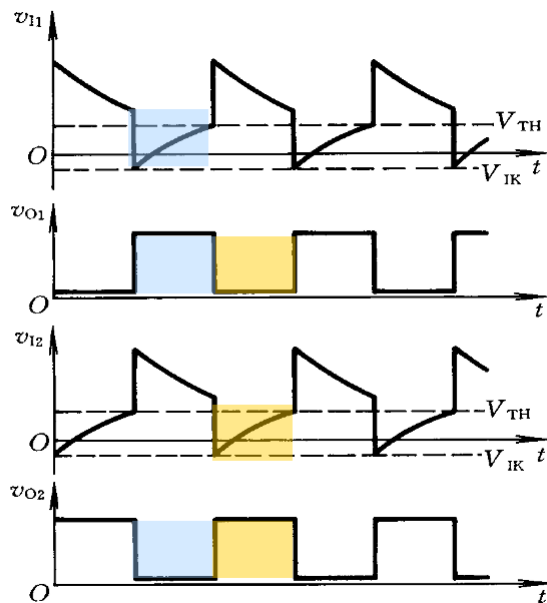
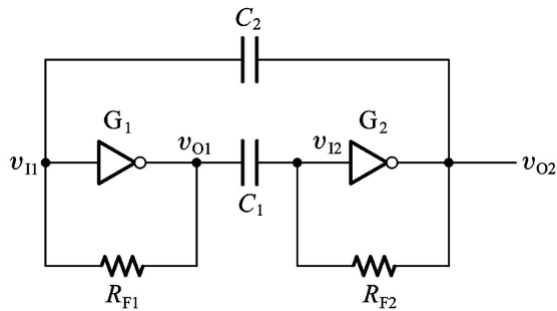


注释：

分析单稳态触发器的稳态可以采用假设法：即假设稳态输出为 0 或者 1，然后结合稳态的条件判断电路是否能维持在该状态；若可以维持则该状态为稳态，否则为暂稳态。

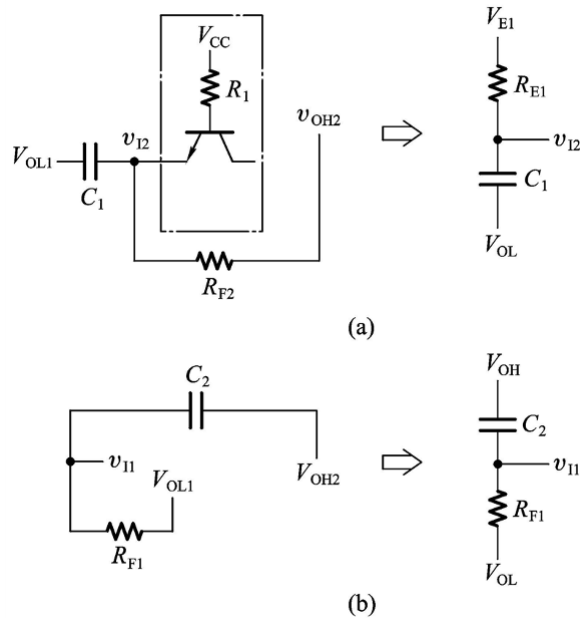
§6.2 多谐振荡器

多谐振荡器是可以自激震荡的电路，不需要外加信号，这里从最简单的对称式多谐振荡器开始分析：



- 当 v_{i1} 上升时, 经过反相器 v_{o1} 下降, 通过 C_1 的耦合作用使得 v_{i2} 下降, 经过反相器 v_{o2} 上升, 通过 C_2 的耦合作用使得 v_{i1} 上升, 形成正反馈
- 当 v_{i1} 下降时, 经过反相器 v_{o1} 上升, 通过 C_1 的耦合作用使得 v_{i2} 上升, 经过反相器 v_{o2} 下降, 通过 C_2 的耦合作用使得 v_{i1} 下降, 形成正反馈

用戴维南定理, 可以将充放电电路进行等效:



其中

$$V_E = V_{OH} + (V_{CC} - V_{BE} - V_{OH}) \frac{R_{F2}}{R_1 + R_{F2}}$$

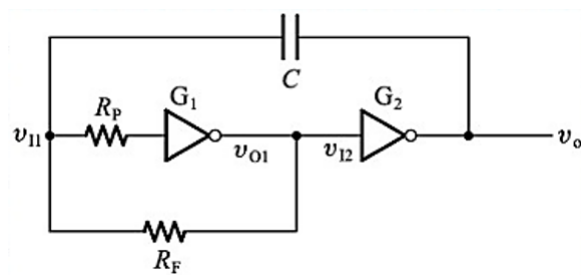
$$R_E = R_1 \parallel R_{F2}$$



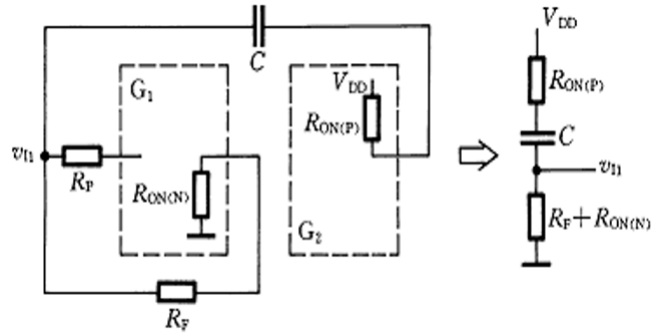
注释:

在分析本题等效电路的时候, 三极管在导通时, 发射极被视作恒压源 V_{BE} , 因此在戴维南定理“独立源置零”的要求下被视作导线; 若三极管没有导通, 则发射极不能再视为恒压源, 而是视为开路。

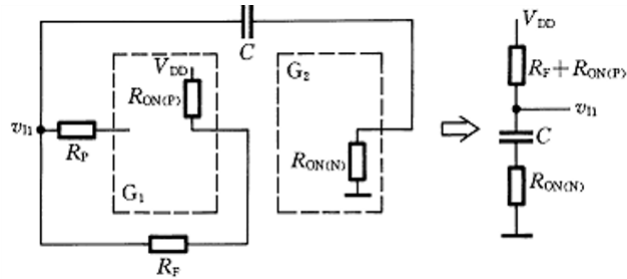
对于非对称式多谐振荡器:



其充放电的等效电路为



$$\begin{cases} v_{I1}(0^+) = 1.5V_{DD} \\ v_{I1}(\infty) = 0 \\ \tau = (R_{ON(P)} + R_f)C \approx R_f C \end{cases}$$



$$\begin{cases} v_{I1}(0^+) = -0.5V_{DD} \\ v_{I1}(\infty) = V_{DD} \\ \tau = (R_{ON(P)} + R_f)C \approx R_f C \end{cases}$$

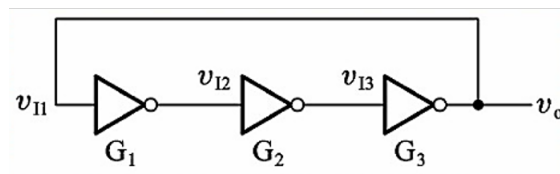


注意：

无论是 TTL 还是 CMOS，输出都不参与计时，一般当做独立的电压源。

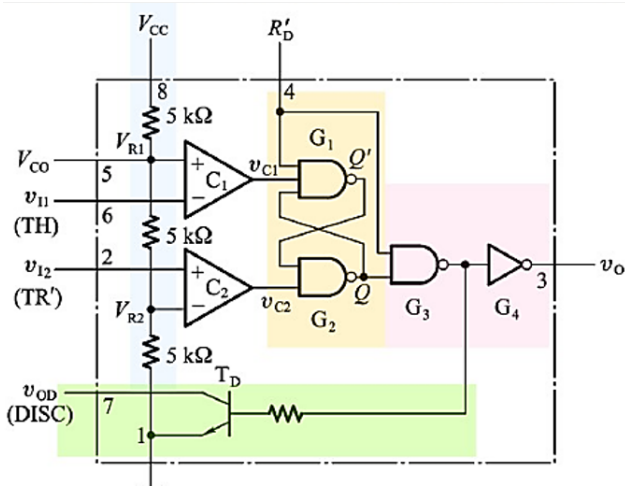
分析时，注意分析电路的初始状态和稳定状态，再进行等效分析。

在上述电路中，我们都没有考虑反相器的传播延时；如果考虑传播延时，则可以用反相器构建环形振荡器：



要注意环形振荡器一定是奇数个反相器，其振荡周期为 $2nt_{pd}$, $n = 2k + 1$ ，否则无法工作。

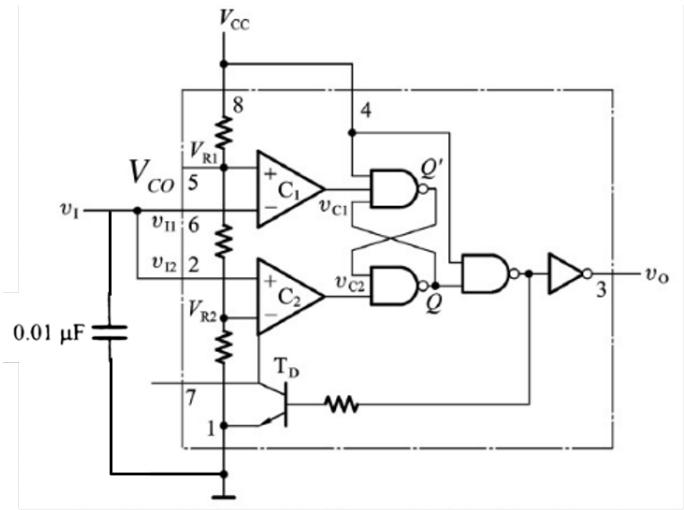
§6.3 555 定时器



555 定时器的功能表如下：

输入			输出	
R'_D	V_{I1}	V_{I2}	V_O	T_D 状态
0	×	×	0	导通
1	$> \frac{2}{3}V_{CC}$	$> \frac{1}{3}V_{CC}$	0	导通
	$< \frac{2}{3}V_{CC}$	$> \frac{1}{3}V_{CC}$	不变	不变
	$< \frac{2}{3}V_{CC}$	$< \frac{1}{3}V_{CC}$	1	截止
	$> \frac{2}{3}V_{CC}$	$< \frac{1}{3}V_{CC}$	1	截止

用 555 定时器，可以做出施密特触发器：



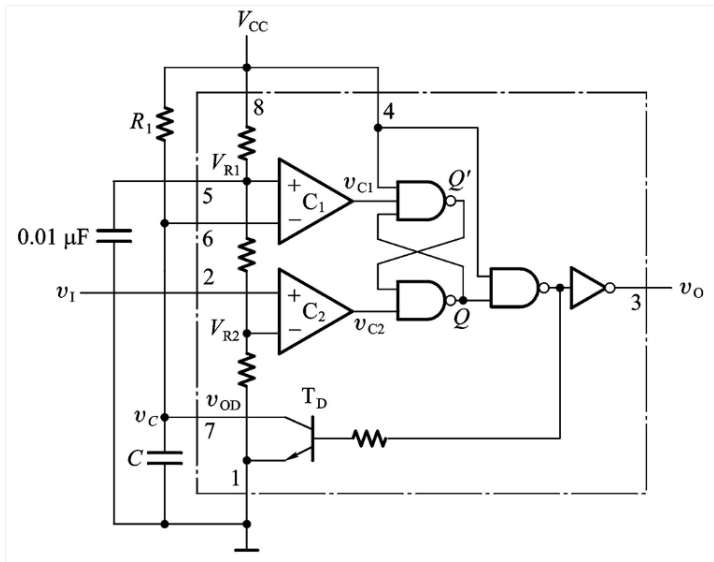


注释:

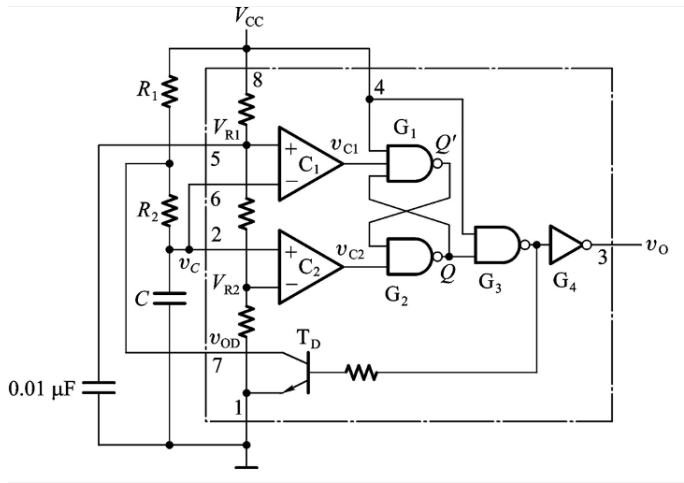
一般在使用 555 计时器的时候都会配合电容使用, 滤除噪声。

该触发器的阈值电压分别为 $\frac{1}{3}V_{CC}$ 和 $\frac{2}{3}V_{CC}$ 。

同样, 也可以做出单稳态触发器:



可以分析, 当 $v_I = 1$ 时, 电路的输出稳态是 $v_O = 0$ 。

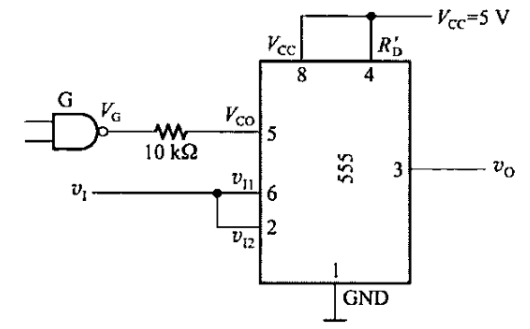


回想起高中通用技术被 555 振荡器支配的恐惧了吗? 该振荡器的反馈电阻为 R_2 , 充电电容为 C ; 充电时, R_1 和 R_2 都在回路中, 放电时, 只有 R_2 在回路中。



例题 6.1

图中是用 555 定时器接成的施密特触发电路。G 为 74HC 系列与非门, 输出电压 V_G 的高、低电平分别为 $V_{OH} = 5V$ 、 $V_{OL} = 0V$, 输出电阻小于 50Ω 。试求 V_G 为高、低电平下电路的 V_{T+} 和 V_{T-} , 并画出电路的电压传输特性。



解：在介绍用 555 定时器接成施密特触发电路的原理时已经讲到，它的正向阈值电压为 $V_{T+} = V_{CO}$ ，负向阈值电压为 $V_{T-} = \frac{1}{2}V_{CO}$ 。因此，只要计算出 V_G 为高、低电平时的 V_{CO} 值，就很容易得到 V_{T+} 和 V_{T-} 了。

V_{CO} 端接在了集成电路内部分压电阻的结点上，如图所示。当 $V_G = 0$ 时，可求得 V_{CO} 为

$$V_{CO} = V_{CC} \frac{R_2 \parallel (R_3 + R_4)}{R_1 + R_2 \parallel (R_3 + R_4)} = 2.5 \text{ V}, \quad V_{T+} = V_{CO} = 2.5 \text{ V}, \quad V_{T-} = \frac{1}{2}V_{CO} = 1.25 \text{ V}$$

当 $V_G = 5 \text{ V}$ 时，利用叠加定理求得 V_{CO} 为

$$V_{CO} = V_{CC} \frac{R_2 \parallel (R_3 + R_4)}{R_1 + R_2 \parallel (R_3 + R_4)} + V_G \frac{R_1 \parallel R_2}{R_3 + R_4 + R_1 \parallel R_2} = 3.74 \text{ V}$$

$$V_{T+} = V_{CO} = 3.74 \text{ V}, \quad V_{T-} = \frac{1}{2}V_{CO} = 1.87 \text{ V}$$



小结：

脉冲电路的分析步骤：

- 分析工作过程→ 画波形→ 找出决定电路状态转换的关键电压
- 画出这点电压充、放电等效电路，化简
 - 打开门电路：对于所有门电路的输出，我们认为是有内阻的电压源
 - TTL 的输入，需要把输入画出来，TTL 的电路取电流，可能会参加充放电
 - CMOS 电路不取电流，因此可以直接断开
- 确定并修订充放电的几个关键值：三要素法
- 计算充放电时间，求出所需要的结果

7 测试电路

定义 7.1 固定型故障模型

假设选定的线信号（门输入或输出）“固定在”逻辑值 0 或 1，并通常假设同一时间只有一个故障。

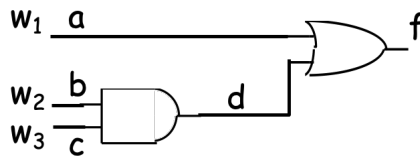


由于假定同一时间只有同一问题，我们可以采取穷举法来求测试集。



例题 7.1

如图所示的电路



其最小的测试向量集 $\{(a, b, c)\} = \{001, 010, 011, 100\}$ （不唯一）。

由于一组组测试耗时耗力，一般会采取激活路径的方法，如固定二输入门的某个输入，使输入门成为（同向或反向）缓冲器，当电路出现问题的时候，一个输入的变化会产生级联效应，因此一个测试可以捕捉多个故障。



小结：

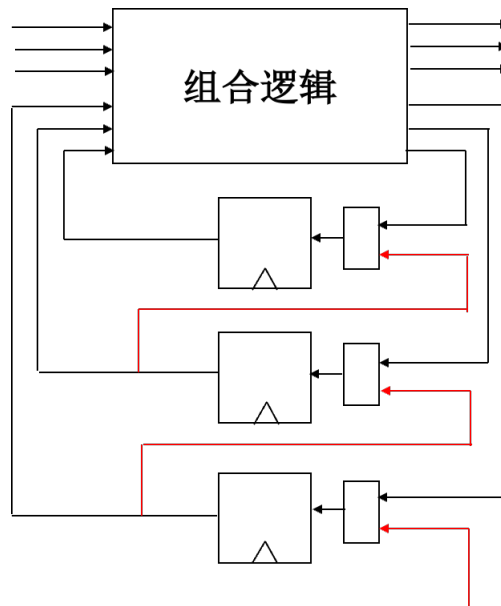
固定型故障模型的测试方法：

1. 激活故障，如线路被固定为 x ，则要求在正常情况下该线路为 $\bar{x}$
2. 传播故障，让故障可以传播到输出，即其它线路的输入不能决定输出（如与门则要求另一个输入为 1，否则直接输出为 0，无法检测故障）

通常来说测试集不是唯一的，而且某些故障可能无法检测（如 $f = a + ab$ ，无论 b 是否发生故障都不会影响电路输出），即使存在最小测试集，寻找最小测试集也是 NP 难问题，因此一般只用取足够小的测试集即可。

如果电路过于复杂，那么可以采取随机测试。一般来说发现故障的概率随测试集的扩大而增长，在现代计算机下几亿个测试向量也用不了多久。

对于时序电路，采用扫描路径技术：触发器的输入经过多路复用器级，允许它们在正常模式和特殊测试移位寄存器模式下使用。



注释:

一般在设计的时候不需要增加扫描路径，有现代工具可以自动加入。

除了扫描测试以外，还可以采用内建自测试的方法：使用反馈移位寄存器生成伪随机序列，从而生成唯一的签名，来确定电路是否正确。



例题 7.2

如下所示代码是多输入特征寄存器 (Multiple Input Signature Register, MISR, 是线性反馈移位寄存器的一种) 的 Verilog 硬件语言描述，请根据此代码画出 MISR 的电路原理图，并且要求设计中不能出现与门（假设器件悬空为低电平）。

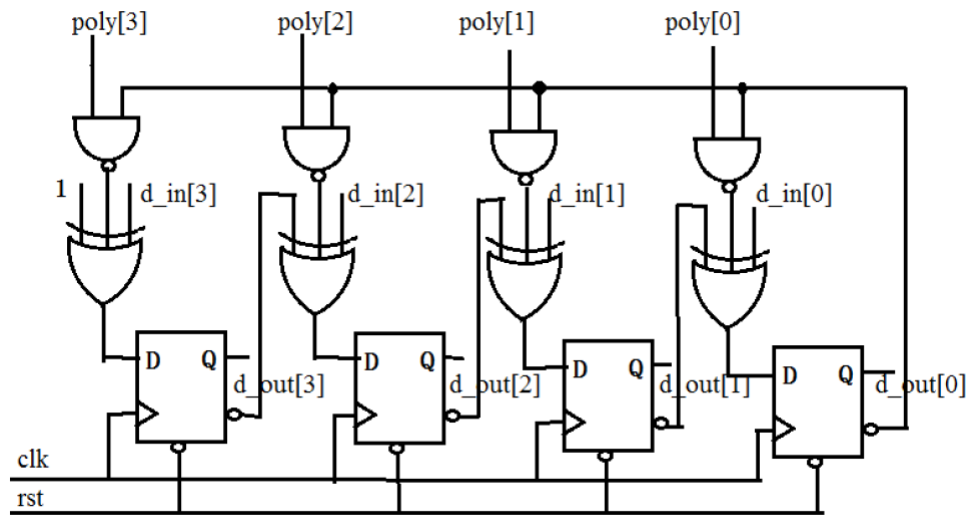
```

1  `timescale 1ns/100ps
2  module misr #(parameter [3:0] poly = 0) (clk, rst, d_in, d_out);
3      input clk;
4      input rst;
5      input [3 : 0] d_in;
6      output [3 : 0] d_out;
7      reg [3 : 0] d_out;
8      always @(posedge clk) begin
9          begin
10             if( rst )
11                 d_out <= 4' b0000;
12             else
13                 d_out <= d_in ^ ({4{ d_out[0]}} & poly) ^ {1' b0 , d_out[3:1] };
14         end

```

15 `endmodule`

解：不能出现与门，则将与门的改为与非门，并将常数 0 改为常数 1。



8 微处理器

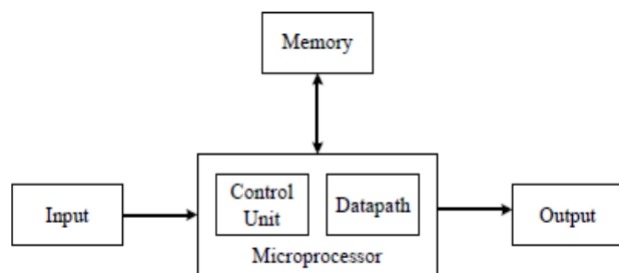
§8.1 控制单元与数据通路

定义 8.1 微处理器

微处理器是一种集成在单一集成电路上的中央处理单元（Central Processing Unit, CPU）。

现代的微处理器主要采取冯·诺依曼结构：

- 将程序存储和数据存储放在同一物理存储空间
- 具有相同的总线
- 使用的硬件结构简单

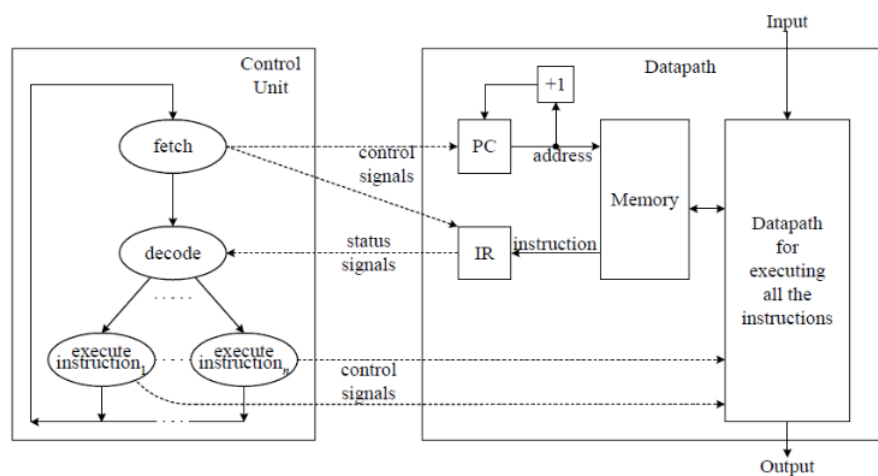


微处理器有两个主要部分：

- 控制单元：用于控制数据通路的所有操作，实现微处理器运算的正确性；
- 数据通路：主要包括运算单元 ALU、存储单元（寄存器）及其相互连接。

定义 8.2 指令集架构

定义了计算机能够执行的指令集合、指令编码格式、操作数的类型，以及对处理器寄存器、存储系统和 I/O 系统的访问方式等。



一般来说微处理器的指令周期有三个执行步骤：

- 取出指令

- 从程序存储器中把 PC 所指向的指令取出，拷贝到指令寄存器 IR
- PC+1 指向下一条指令的地址
- 编译指令
 - 从 IR 中提取出操作码，对指令所要做的操作进行翻译
 - 决定当前执行那一条指令，跳转到那一个对应的状态去
- 执行指令
 - 每一步将在有限状态机的一个状态被执行
 - 每个指令通常在一个时钟周期内执行



注意：

有些情况下一些存储器操作的指令可能需要两个甚至更长的时钟周期，这种情况下就需要更多的状态来实现正确的时序。

现代的许多计算机也不一定可以在三个时钟周期内完成一次指令执行，需要视情况而定。

§8.2 机器语言与汇编语言

你不要问我为什么数电还要学汇编语言。

- 加法 $a = b + c$ ，汇编 `add a, b, c`
- 减法 $a = b - c$ ，汇编 `sub a, b, c`

MIPS 寄存器名称由 \$ 符号开头，如 `$s0 = a` 表示把 a 放在 s0 寄存器里。

名称	编号	用途
<code>\$0</code>	0	常数值 0
<code>\$at</code>	1	汇编器临时寄存器
<code>\$v0-\$v1</code>	2-3	过程返回值
<code>\$a0-\$a3</code>	4-7	过程参数
<code>\$t0-\$t7</code>	8-15	临时变量
<code>\$s0-\$s7</code>	16-23	保存变量
<code>\$t8-\$t9</code>	24-25	临时变量
<code>\$k0-\$k1</code>	26-27	操作系统临时寄存器
<code>\$gp</code>	28	全局指针
<code>\$sp</code>	29	栈指针
<code>\$fp</code>	30	帧指针
<code>\$ra</code>	31	过程返回地址

字节寻址有两种方式：

- 大端寻址：字节的编号 0 从大（most significant）端开始
- 小端寻址：字节的编号 0 从小（least significant）端开始

立即数操作： $a = a + 4$ ，汇编 `$s0 = a ; addi $s0, $s0, 4`

然后我更不知道为什么还要学机器语言。

- R 型语言：寄存器类型，32 位指令

- op, 也称为 opcode 或操作码, R 型指令的 opcode 为 6'b000000
- rs, rt, 源寄存器
- rd, 目标寄存器
- shamt, 只用于移位操作, 其它 R 型指令, shamt 为 5'b00000
- funct, 也称为功能码, 确定特定的 R 型操作
- I 型语言: 立即数类型, 32 位指令
 - op, 操作码
 - rs, 源操作数
 - rt, 如 addi 和 lw 用作目标操作数, sw 作为另一种源操作数
 - imm, 源操作立即数, rs 和 imm 始终用作源操作数



注释:

$$\text{无符号数: } value = \sum_{i=0}^{n-1} b_i 2^i, \text{ 有符号数: } value = -(b_{n-1} 2^n) + \sum_{i=0}^{n-1} b_i 2^i$$

本节内容作为基础性概念理解, 更多内容参考《计算机组成与设计》。

9 Verilog 语言

§9.1 Verilog



注意:

Verilog 语言是大小写敏感的语言, 大小写的不同会导致变量不同甚至编译不通过。

module 用于描述电路的一个模块; **wire** 用于表示物理连线。



例题 9.1

异或门的 Verilog 描述。

```

1  module xor_gate ( out, a, b );           //模块名称与接口端
2      input      a, b;                     //声明输入
3      output     out;                      //声明输出
4      wire       abar, bbar, t1, t2;      //变量定义
5
6      inverter   invA (abar, a);           //内建非门, 格式为(反变量, 原变量)
7      inverter   invB (bbar, b);
8      and_gate   and1 (t1, a, bbar);      //内建与门, 格式为(输出, 输入1, 输入2)
9      and_gate   and2 (t2, b, abar);
10     or_gate     or1  (out, t1, t2);      //内建或门, 格式为(输出, 输入1, 输入2)
11
12 endmodule

```



例题 9.2

二选一数据选择器的 Verilog 描述。

```

1  //2-input multiplexer in gates
2  module mux2 (in0, in1, select, out);
3      input in0,in1,select;
4      output out;
5      wire s0,w0,w1;
6
7      not (s0, select);                    //对select信号取反
8      and (w0, s0, in0),
9          (w1, select, in1);
10     or  (out, w0, w1);                    //最后的输出是或门

```

```

11
12 endmodule // mux2

```

assign 语句用于连续赋值。

```

1 module and_gate (out, in1, in2);
2     input    in1, in2;
3     output   out;
4
5     assign out = in1 & in2; //输入变化会连续导致输出变化
6
7 endmodule

```

reg 用于存储变量；**always** 语句用于过程块赋值语句。

```

1 // Behavioral model of 2-to-1
2 // multiplexer.
3 module mux2 (in0,in1,select,out);
4     input in0,in1,select;
5     output out;
6     //
7     reg out;
8     always @ (in0 or in1 or select)
9         if (select) out=in1;
10        else out=in0;
11 endmodule // mux2

```



注释:

新的语法支持 **always @ (*)**，可以避免因敏感列表未列全带来的问题。

input [1:0] 可以定义一个二维向量。



例题 9.3

四选一数据选择器的 Verilog 描述。

```

1 //4-input mux behavioral description
2 module mux4 (in0, in1, in2, in3, select, out);
3     input in0,in1,in2,in3;
4     input [1:0] select;
5     output   out;
6     reg      out;

```



```

7
8     always @ (in0 in1 in2 in3 select)
9         case (select)
10             2'b00: out=in0;
11             2'b01: out=in1;
12             2'b10: out=in2;
13             2'b11: out=in3;
14         endcase
15 endmodule // mux4

```

{A, B} 可以 A 和 B 拼成二维向量进行赋值。



例题 9.4

四 bit 全加器的 Verilog 描述。

```

1 module full_addr (S, Cout, A, B, Cin);
2     input  A, B, Cin;
3     output S, Cout;
4
5     assign {Cout, S} = A + B + Cin; //S表示结果, Cout用于表示进位
6 endmodule
7
8 module adder4 (S, Cout, A, B, Cin);
9     input  [3:0] A, B;
10    input      Cin;
11    output [3:0] S;
12    output      Cout;
13    wire      C1, C2, C3;
14
15    full_addr fa0 (S[0], C1,  A[0], B[0], Cin);
16    full_addr fa1 (S[1], C2,  A[1], B[1], C1);
17    full_addr fa2 (S[2], C3,  A[2], B[2], C2);
18    full_addr fa3 (S[3], Cout, A[3], B[3], C3);
19 endmodule

```

Verilog 有四种基本常量:

- 0, 1 分别表示低电平和高电平
- x 表示不关心或不确定的变量
- z 表示高阻态

变量可以写成向量的形式，如 `A[3:0]` 表示四位向量 `{A[3], A[2], A[1], A[0]}`。

Verilog 可以采用多种进制，如 `12'b` 表示 12 位二进制，`12'h` 表示 12 位十六进制；如果数字太长可采用下划线进行分段，编译时会忽略。



注意：

Verilog 变量是无符号的，如 `A = 0110` (6) 而 `B = 1010` (-6)，则 `A + B = 10000`；`reg` 应当只与 `always` 语句块使用，不要与 `assign` 语句混用。

模块的端口 (port) 可以声明为三种类型：

- `input` 声明为输入
- `output` 声明为输出
- `inout` 声明既是输入又是输出



注释：

Verilog 语言的关键字都是小写的，但部分系统（如 Windows）是不区分大小写的，因此建议模块名小写，输入输出的变量首字母大写。

可以使用 `assign #n` 进行连续 `n` 次赋值，但是不建议使用。



例题 9.5

数据比较器的 Verilog 描述。

```

1  module Compare1 (A, B, Equal, Alarger, Blarger);
2      input    A, B;
3      output   Equal, Alarger, Blarger;
4
5      assign Equal = (A & B) | (~A & ~B);
6      assign Alarger = (A & ~B);
7      assign Blarger = (~A & B);
8  endmodule
9
10 // Make a 4-bit comparator from 4 1-bit comparators
11 module Compare4(A4, B4, Equal, Alarger, Blarger);
12     input  [3:0] A4, B4;
13     output Equal, Alarger, Blarger;
14     wire    e0, e1, e2, e3, A10, A11, A12, A13, B10, B11, B12, B13;
15
16     Compare1 cp0(A4[0], B4[0], e0, A10, B10);
17     Compare1 cp1(A4[1], B4[1], e1, A11, B11);

```

```

18    Compare1 cp2(A4[2], B4[2], e2, A12, B12);
19    Compare1 cp3(A4[3], B4[3], e3, A13, B13);
20    //Verilog是并行处理的,所以上述四句语言是同时执行的
21
22    assign Equal    = (e0 & e1 & e2 & e3);
23    assign Alarger  = (A13 | (A12 & e3) |
24                      (A11 & e3 & e2) |
25                      (A10 & e3 & e2 & e1));
26    assign Blarger  = (~Alarger & ~Equal);
27    endmodule

```

在 `always` 过程块里,被赋值的变量一定是 `reg` 类型。

§9.2 逻辑综合

Verilog 语言中有许多的操作符,大多数与 C 语言类似,下面给出几种 C 语言不常用或不存在的操作符:

- 逻辑操作符: `&&`, `||`, `!`
- 位操作符: `&`, `|`, `~`(非), `^`(异或), `~^`(同或)
- 位移操作符: `<<`(左移,低位补 0), `>>`(右移,高位补 0)
- 拼接操作符: `{a, b}`(将 `a` 和 `b` 拼接为一个新的向量)



例题 9.6

数据选择器

```

1  module foo (a,b,s0,s1,f);
2  input [3:0] a;
3  input [3:0] b;
4  input s0,s1;
5  output [3:0] f;
6  reg f;
7  always @ (a or b or s0 or s1)
8      if (!s0 && s1 || s0) f = a; else f = b; //条件等价于s1||s0
9  endmodule

```

对于一般的 Verilog 代码,我们一般按照以下的模板进行编写,以方便共同开发:

```

1  module <top_module_name>(<port list>);
2  /* Port declarations. followed by wire, reg, integer, task and function
   declarations */
3  /* Describe hardware with one or more continuous assignments, always blocks,
   module instantiations and gate instantiations */

```

```

4 // Continuous assignment
5 wire <result_signal_name>;
6 assign <result_signal_name> = <expression>;
7 // always block
8 always @( <event expression> )
9 begin
10 // Procedural assignments
11 // if statements
12 // case, casex, and casez statements
13 // while, repeat and for loops
14 // user task and user function calls
15 end
16 // Module instantiation
17 <module_name> <instance_name> (<port list>);
18 // Instantiation of built-in gate primitive
19 gate_type_keyword (<port list>);
20 endmodule

```

Verilog 中赋值语句分两大类：

- 阻塞赋值：=，如 $A = 1$, $A = 2$, $B = A$ ，则 $A = 2$, $B = 2$
- 非阻塞赋值：<=，如 $A = 1$, $A <= 2$, $B <= A$ ，则 $A = 2$, $B = 1$ ，可以理解为先寄存赋值，在过程块全部执行完成后同步发生



注意：

不要在一个 **always** 模块里同时混合使用阻塞赋值和非阻塞赋值，容易造成仿真结果与实际结果不同。

如 $d <= e$, $d = f$ ，则 d 的值取决于仿真软件的调度顺序，最终 d 的取值可能是 e 也可能是 f ，在电路实现上会出问题。

Verilog 中只有一个三目操作符 $A ? B : C$ ，若 A 为真，则得到表达式 B 的值，否则得到 C 的值。



例题 9.7

分析下面这段代码：

```

1 module mux4to1 (out, a, b, c, d, sel);
2 output out;
3 input a, b, c, d;
4 input [1:0] sel;
5 reg out;
6 always @(sel or a or b or c or d)
7 begin

```

```

8     case (sel)
9         2'd0: out = a;
10        2'd1: out = b;
11        2'd3: out = d;
12    endcase
13 end
14 endmodule

```

该代码下板运行时会出现问题：当 `sel = 2` 时，无对应分支，因此仿真器会推断出该电路为锁存器，输出为上一次输出的值，而不是期望的值。

修正方法：

1. 补全分支：`2'd2: out = c;`
2. 设定默认值：`default: out = c;`
3. 设置无关态：`default: out = 1'bx;`（在实际工程中会造成输出不确定，因此慎用）

Verilog 中，`if-else` 语句是串行处理的，而 `case` 语句是并行处理的，因此用两者编写的硬件行为并不完全一样（如使用 `if-else` 容易生成优先编码器）。



例题 9.8

同步 D-边沿触发器：

```

1 module dff(q, d, clk, set, rst);
2     input d, clk, set, rst;
3     output q;
4     reg q;
5
6     always @(posedge clk)
7         if (rst)
8             q <= 0;
9         else if (set)
10            q <= 1;
11        else
12            q <= d;
13 endmodule

```

对于一个常见的有限状态机（FSM），我们可以用下面的模板进行描述。

```

1 module FSM1 (clk, rst, enable, data_in, data_out);
2     input clk, rst, enable;
3     input data_in;
4     output data_out;

```

```
5
6 // 状态编码
7 parameter IDLE = 2'b00;
8 parameter READ = 2'b01;
9 parameter WRITE = 2'b10;
10 parameter INVALID = 2'b11;
11
12 reg data_out;
13 reg [1:0] state, next_state;
14
15 // 时序逻辑：状态更新
16 always @(posedge clk) begin
17     if (rst)
18         state <= IDLE;
19     else
20         state <= next_state;
21 end
22
23 // 组合逻辑：输出和下一状态
24 always @(*) begin
25     case (state)
26         IDLE: begin
27             data_out = 1'b0;
28             if (enable)
29                 next_state = READ;
30             else
31                 next_state = IDLE;
32         end
33         READ: begin
34             data_out = 1'b1;
35             if (data_in)
36                 next_state = WRITE;
37             else
38                 next_state = IDLE;
39         end
40         WRITE: begin
41             data_out = 1'b0;
42             next_state = IDLE;
43         end
44     endcase
45 end
```

```
44         INVALID: begin
45             data_out = 1'bx;    // 不定态输出
46             next_state = IDLE; // 恢复安全状态
47         end
48         default: begin        // 防止未覆盖的编码（如 2'bxx）
49             data_out = 1'bx;
50             next_state = IDLE;
51         end
52     endcase
53 end
54 endmodule
```



注意:

以上只是 Verilog 的冰山一角，请自行打代码验证。



小结:

关于 Verilog 语言，有一门《数字系统设计实验》课程专门进行训练，因此在本理论课程上考核难度较低，一般为读代码和改错题。

期末复习

逻辑代数

- 基本逻辑公式（离散数学）
- 最大项之积与最小项之和
- 卡诺图化简法（主要是四变量）

组合逻辑

- 编码器与译码器（具体芯片具体分析，尤其注意高低电平有效的问题）
- 数据选择器（ n 位地址可实现 $n + 1$ 位的逻辑函数）
- 半加器与全加器，数据比较器
- 芯片的级联与控制
- 竞争-冒险（成因与消除方法）

时序逻辑

- 锁存器与触发器（RS, JK, T, D），特征方程
- 有限状态机（Moore 与 Mealy 型）
- 移位寄存器，计数器

控制逻辑

- 存储器，可编程逻辑阵列
- ASM 图，状态转换图
- 微指令与微程序控制器（指令设计与结构框图）

门电路

- 三极管电路的放大与饱和
- 计算电阻与输入输出接口数

脉冲电路

- 一阶电路分析（三要素法则），积分电路与微分电路
- 施密特触发器与单稳态触发器，多谐振荡器
- 555 定时器（阈值电压），555 电路

测试电路

- 固定型故障模型（同一时间只有一个测试故障）
- 内建自测试与时序测试

微处理器

- 冯·诺依曼结构域哈佛结构
- 控制单元与数据通道

Verilog

- 基本数据类型（wire 和 reg）
- 赋值语句（assign 连续赋值和 always 过程块），阻塞赋值与非阻塞赋值
- 操作符与关键字

这个就是和大家说明一下。——lp